BIT.MLMR

Symbolic AI: Search and Problem Solving



Lecture 4







Outline

- What is search?
- Problem Representation
  - *State Space* and *And/Or Graph*
  - They embody two thoughts of problem solving!
- Graph Search
  - General graph search algorithm
  - *Blind (uninformed) search*
  - *Heuristic (informed) search*
- Adversarial Search
  - *Minimax algorithm*
  - *α-β pruning*
  - *Monte Carlo Tree Search (MCTS)*

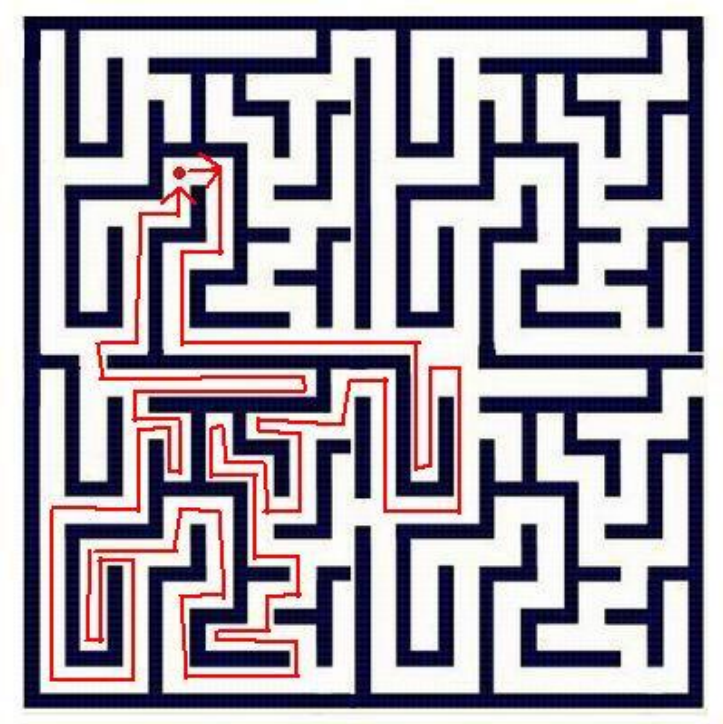
Part I : What is search?

Theseus and the Minotaur



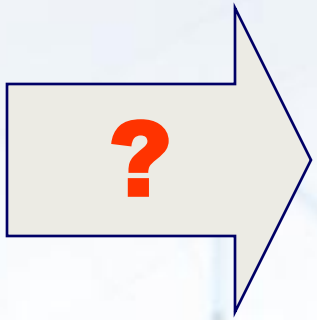
How does Theseus find the way out of Minotaur's labyrinth?

Ariadne's clew:

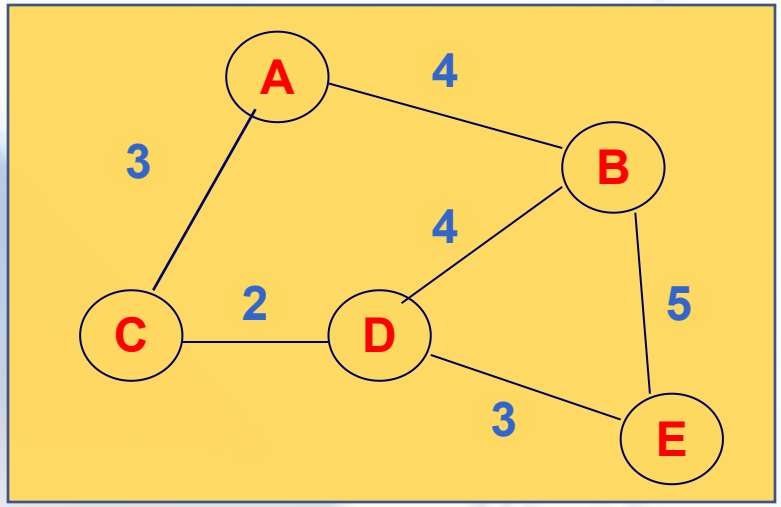


Search Problem Examples

2	8	3	7
10	5	14	1
15		9	6
4	11	14	12



1	2	3	4
8	7	6	5
9	10	11	12
	15	14	13





What is search ?

- With acceptable ***costs***, find out an optimal or feasible ***solution*** in all the possible solutions of the problem
- Ideal search algorithm: find out the optimal solution as soon as possible.
- ***Effectiveness*** and ***Efficiency*** are contradictions to one another

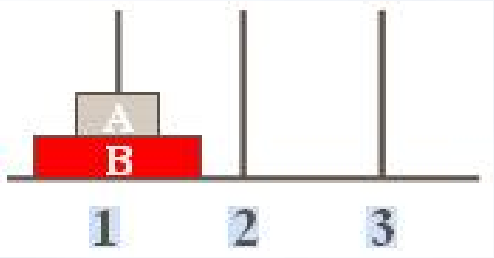
  - Completeness, Optimality, Complexity

  - Blind search vs. Heuristic search； Global search vs. Local search； Feasible solution vs. Optimal solution

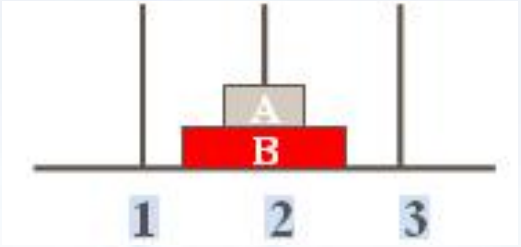
Search Methods

- **Deterministic**
  - Calculus-based,
  - Greedy,
  - Hill-climbing
  - ….
- **Stochastic**
  - Random,
  - Simulated annealing,
  - Evolutionary computing,
  - Swarm intelligence,
  - …..

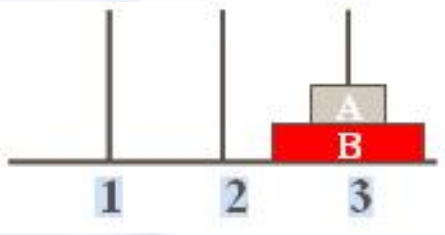
Search Problem Illustration: How to solve 2-rank Hanoi problem?



Initial State



Goal State 1



Goal State 2

Rules:

1. Move only one plate each time
2. The bigger plate must not be placed on the smaller plate.

How to solve 2-rank Hanoi problem?

Step1 Represent the problem, here the state space method is used.

Step 1-1. Represent all the states, including initial states and goal states, and then find a way from initial states to goal states.

$$S_k = \{A_k, B_k\}$$

Initial state

$$S_0 = \{1,1\} \quad S_1 = \{1,2\} \quad S_2 = \{1,3\}$$
$$S_3 = \{2,1\} \quad S_4 = \{2,2\} \quad S_5 = \{2,3\}$$
$$S_6 = \{3,1\} \quad S_7 = \{3,2\} \quad S_8 = \{3,3\}$$

Goal states

How to solve 2-rank Hanoi problem?

Step1-2 Represent the successor functions

$A(i,j)$ Move plate A from pole *i* to pole *j*

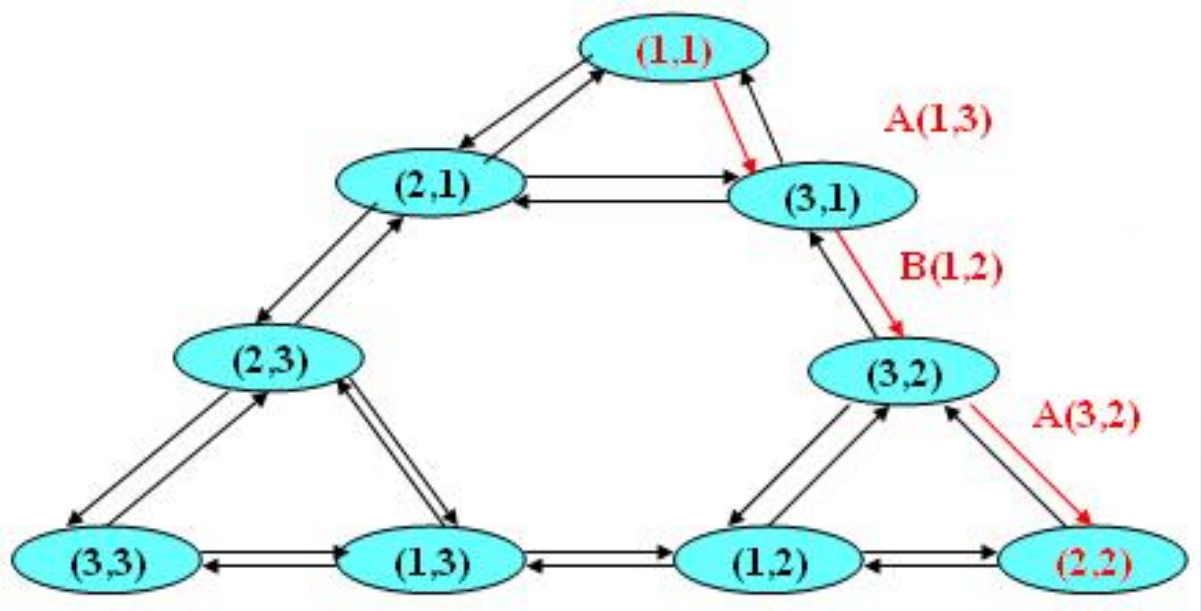
$B(i,j)$ Move plate B from pole *i* to pole *j*

All functions:

$$A(1,2)\quad A(1,3)\quad A(2,1)\quad A(2,3)\quad A(3,1)\quad A(3,2)$$
$$B(1,2)\quad B(1,3)\quad B(2,1)\quad B(2,3)\quad B(3,1)\quad B(3,2)$$

How to solve 2-rank Hanoi problem?

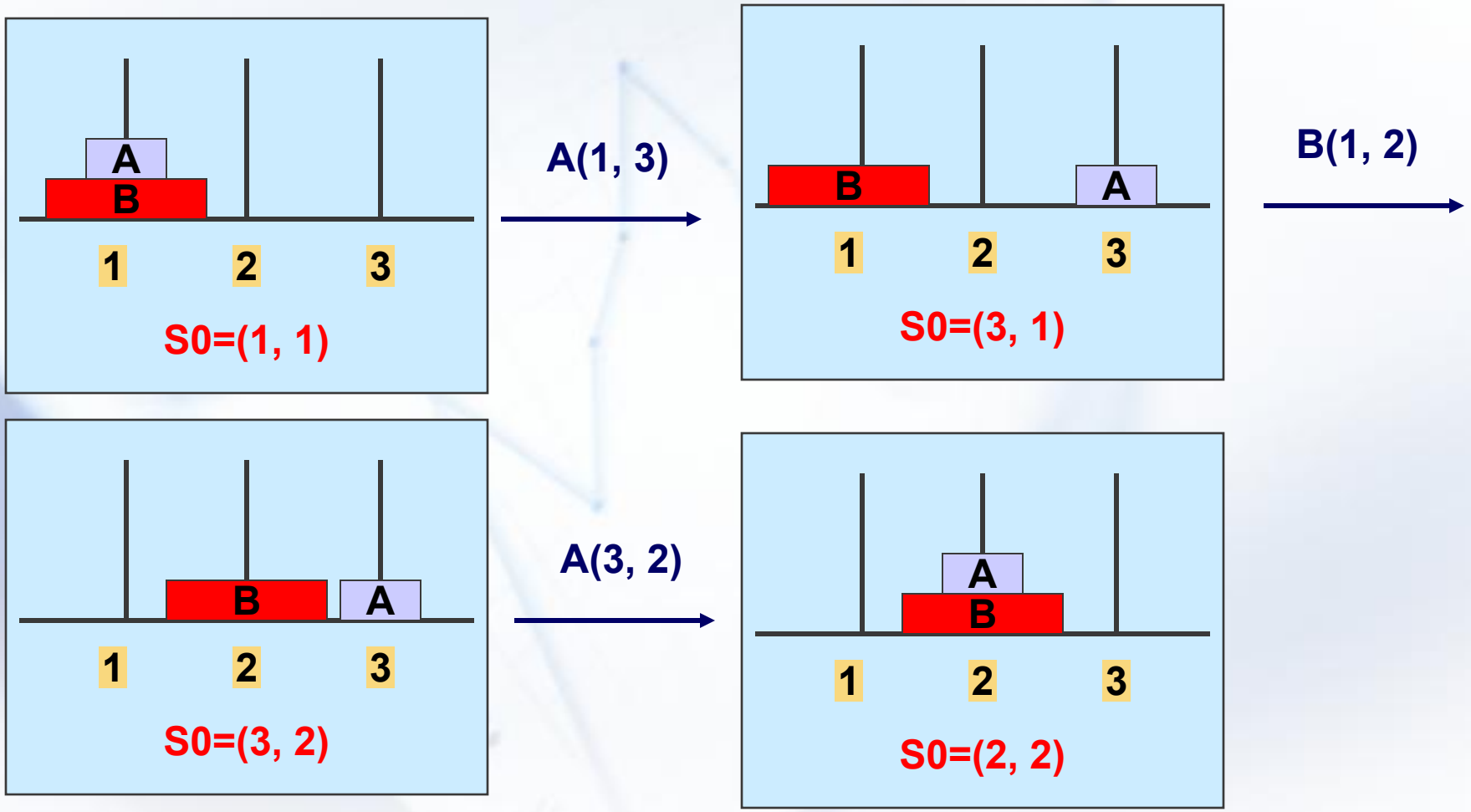
Step1-3 Construct the state space



Step2 Search the graph for the solution

How to solve 2-rank Hanoi problem?

Result:



Part Ⅱ: Problem Representation

2.1 State Space

$$(S, F, C, I, G)$$

- ***S*: the set of states**
- ***F*: the set of successor functions (or actions) used to transform states**
- ***C*: the set of function costs (dispensable if the optimal solution is not concerned)**
- ***I*: the set of initial states**
- ***G*: the set of goal states**

A state space is corresponding with a graph, in which a path from a initial state to a goal state is a solution to the problem

State Space based Problem Solving

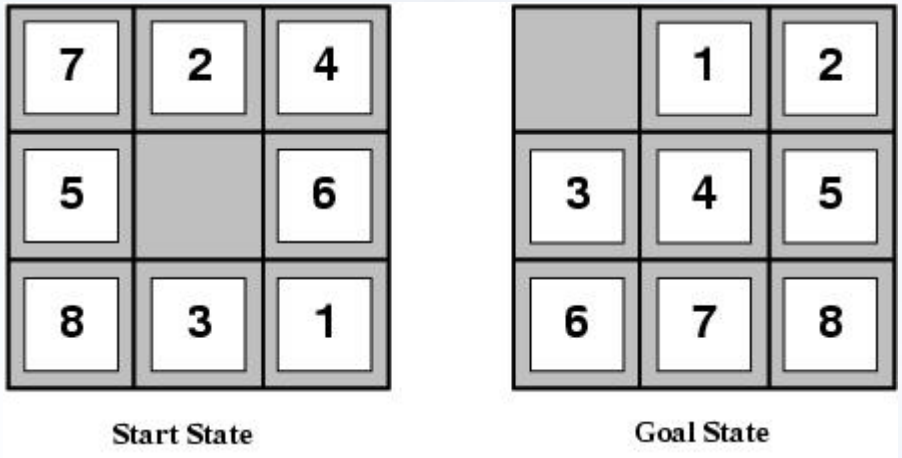
Step1 Represent all states, in which the initial states and goal states are labeled.

Step2 Represent all successor functions

Step3 Take states as node and successor functions as edge to get the corresponding graph

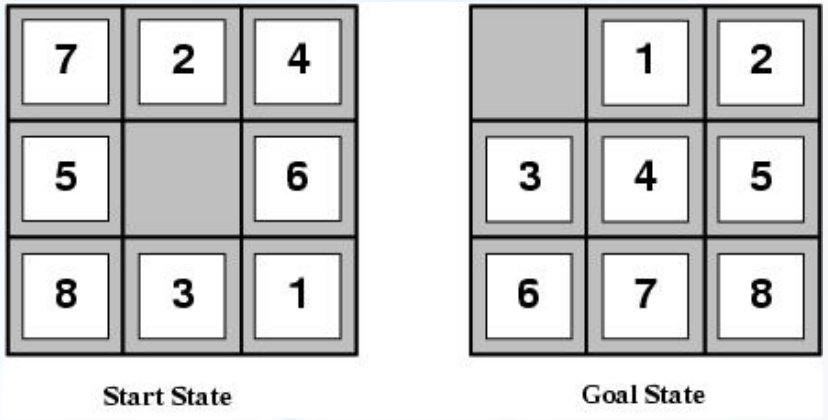
Step4 Search the graph for optimal or feasible solution (path)

Example: The 8-puzzle



- states?
- actions?
- Initial and goal states?
- action costs?

The 8-puzzle



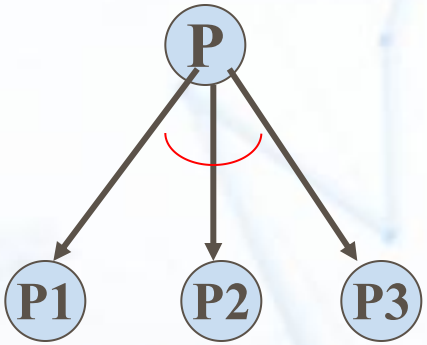
- **states?** locations of tiles
- **actions?** move blank left, right, up, down
- **Initial and goal states?** given
- **action costs?** 1 per move

[Note: optimal solution of *n*-Puzzle family is NP-hard]

2.2 Problem Reduction

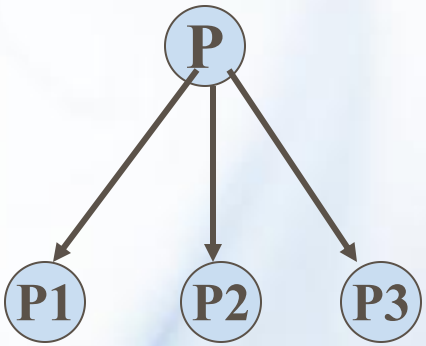
- Solve the difficult problem through solving the equivalent but simpler problem(s).
- Two methods of problem Reduction

"and"



Decomposition

"or"



Transformation

And/Or Graph

- **Data structure for problem reduction**
  - Each node is corresponding with a problem
  - "And" node = decomposition
  - "Or" node = transformation
  - End nodes
    - + Stop nodes = Primitive problems
    - + Other end nodes = Unsolvable problems
  - Solvable nodes and Unsolvable nodes

And/Or Graph based Problem-Solving

- **To find the solution graph which is a sub-graph representing the solution to the original problem**

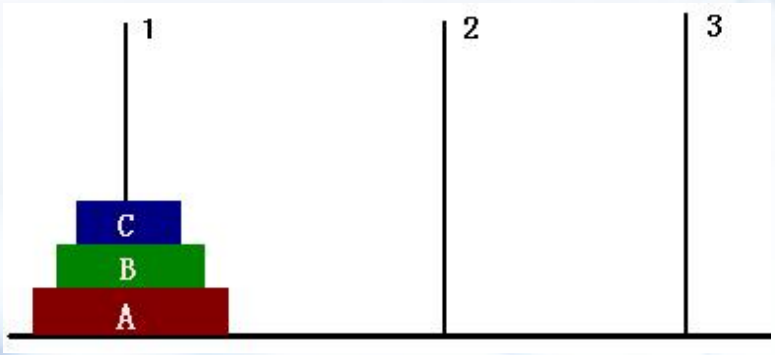
Step1. Represent each problem

Step2. Reduce the problem and represent the reduction process using and/or graph

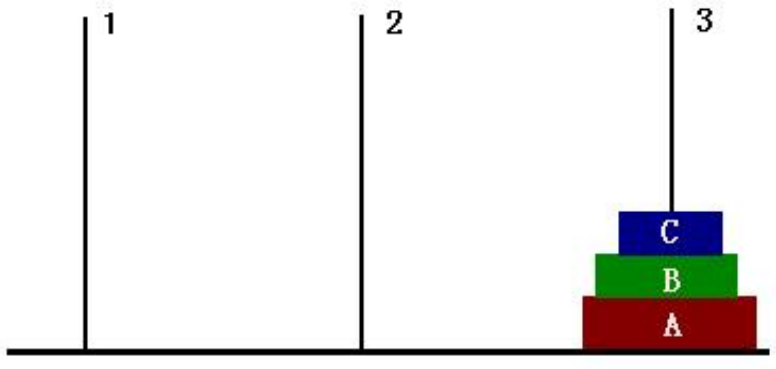
Step3.Backtrack from end nodes to the root node, labeling nodes solvable or unsolvable

Step4. Output the corresponding solution graph if the root node is labeled solvable.

Example: 3-rank Hanoi problem



Initial State



Goal State

3-rank Hanoi problem

Step1. Represent problems

Problem states: (A_k, B_k, C_k)

Original problem: (1,1,1)→(3,3,3)

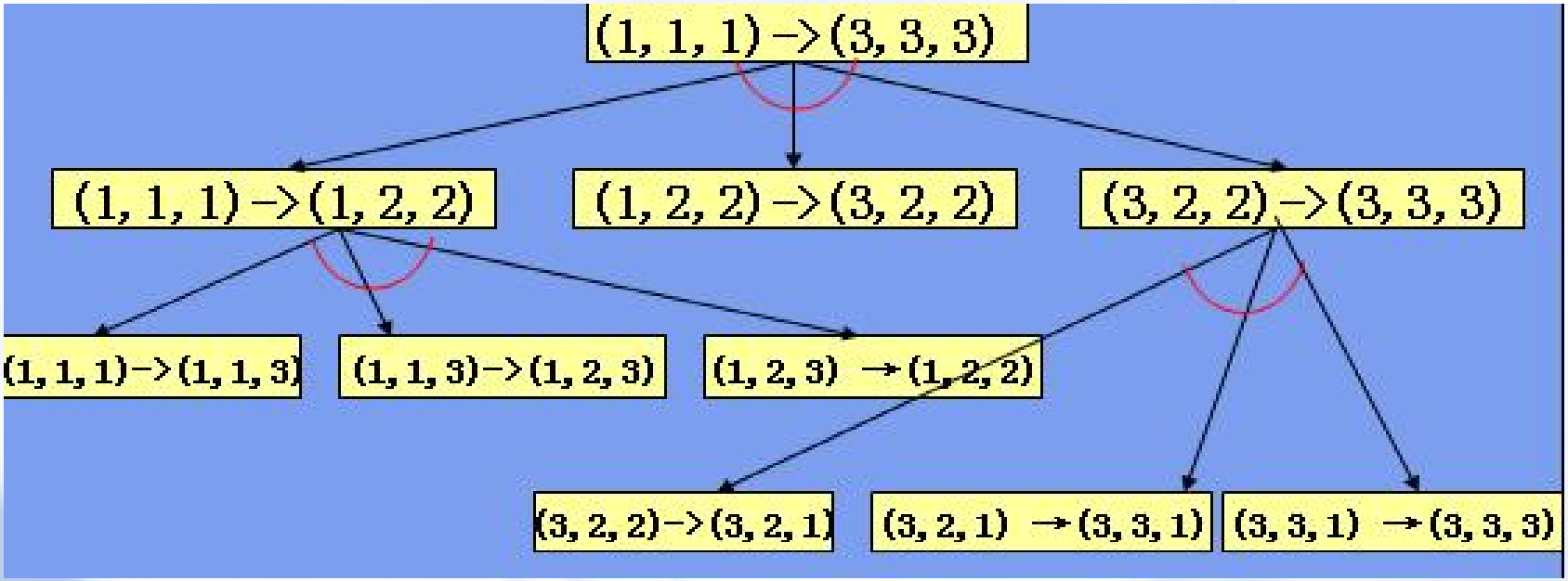
Step2. Problem Reduction

Decompose the original problem into:

- **sub-problem 1 = (1,1,1)→(1,2,2)**
- **sub-problem 2 = (1,2,2)→(3,2,2)** √
- **sub-problem 3 = (3,2,2)→(3,3,3)**

Continue to decompose sub-problem 2 and 3 ……

3-rank Hanoi problem



Step3. Search the graph to determine the root node solvable or unsolvable.

Step4. Output the whole graph as the solution. (how to explain it ?)

Part Ⅲ: Graph Search

3.1 General graph search

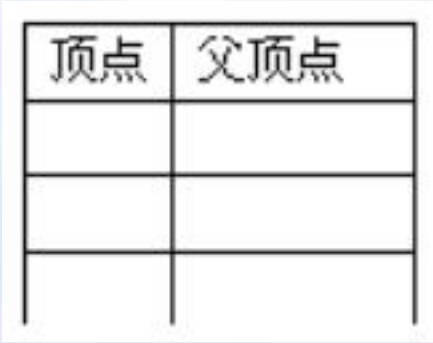
- **Basic idea**

  Gradually generate the graph through expanding nodes in the search process.

- **Basic data structure**

  – Open table: save those searched but unexpanded nodes

  – Closed table: save expanded nodes



General graph search algorithm

Step1. Put the initial node into open table

Step2. Select a node from open table and put it into closed table

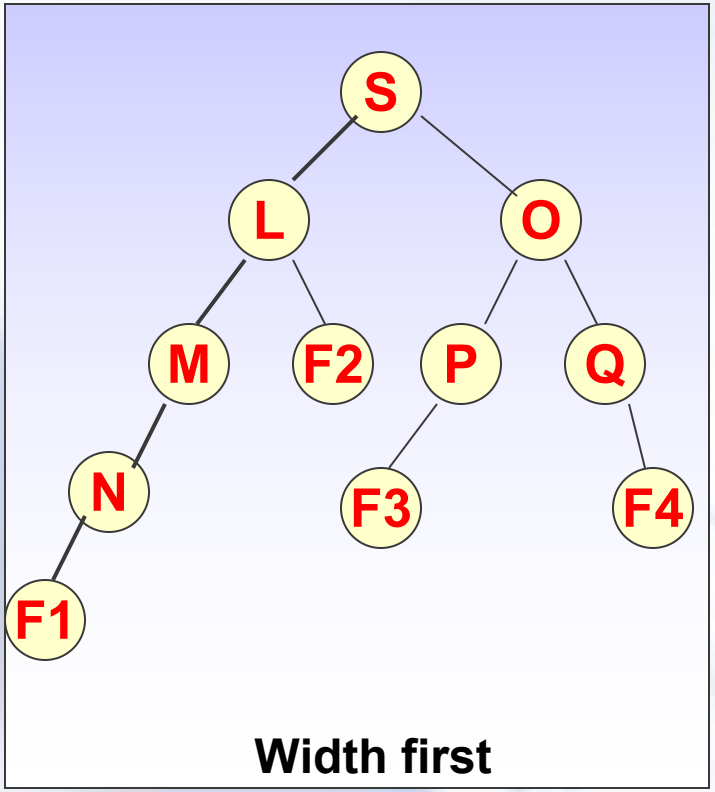
Step3. Expand the selected node: (blue: only for and/or graph)

- If the node can be expanded: put all the expanded nodes into open table and backtrack if needed
- If the node cannot be expanded: Backtrack for determining whether ancestor nodes are unsolvable.

Step4. Repeat Step2-3 until:

- the goal node is found out, or open table is empty (for state space)
- the initial node is labeled solvable or unsolvable, or open table is empty (for and/or graph)

An Illustration of Graph Search



	OPEN	CLOSED
1.	S	
2.		S
3.	LO	S
4.	O	SL
5.	OMF2	SL
6.	MF2	SLO
7.	MF2PQ	SLO
8.	F2PQ	SLOM
9.	F2PQN	SLOM
10.	PQN	SLOMF2
11.	QN	SLOMF2P
12.	QNF3	SLOMF2P
13.	NF3	SLOMF2PQ
14.	NF3F4	SLOMF2PQ
15.	F3F4	SLOMF2PQN
16.	F3F4F1	SLOMF2PQN

g

Key Factor

Selection method in Step 2 is the key difference between different search strategies:

- Blind search vs. Heuristic search

  Select the expanding node without or with consideration of the particularity of the problem

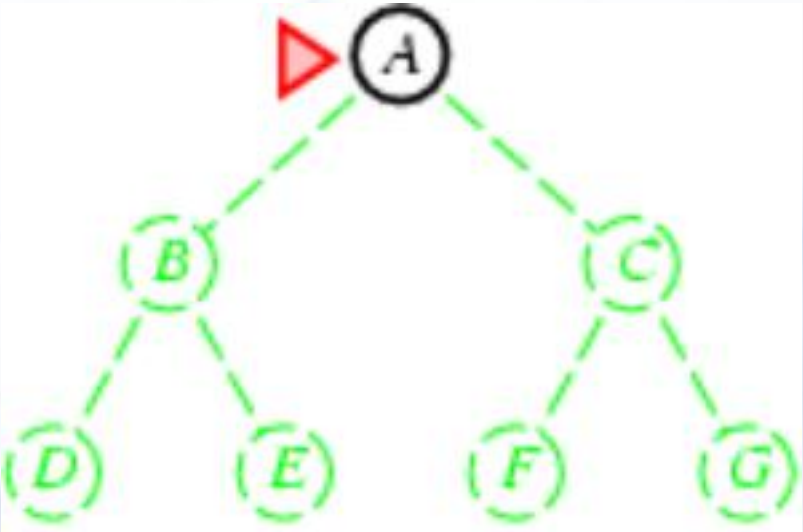
3.2 Blind search strategies

- **Breadth-first search (BFS)**
  - First In First Out (FIFO)
- **Depth-first search (DFS)**
  - Last In First Out (LIFO)
- **Depth-limited search (DLS)**
  - Depth-first search with limited depth
- **Iterative deepening search (IDS)**
  - Iterative depth-limited search with increased depth-limitation

(1) Breadth-first search

- **Expand shallowest unexpanded node**

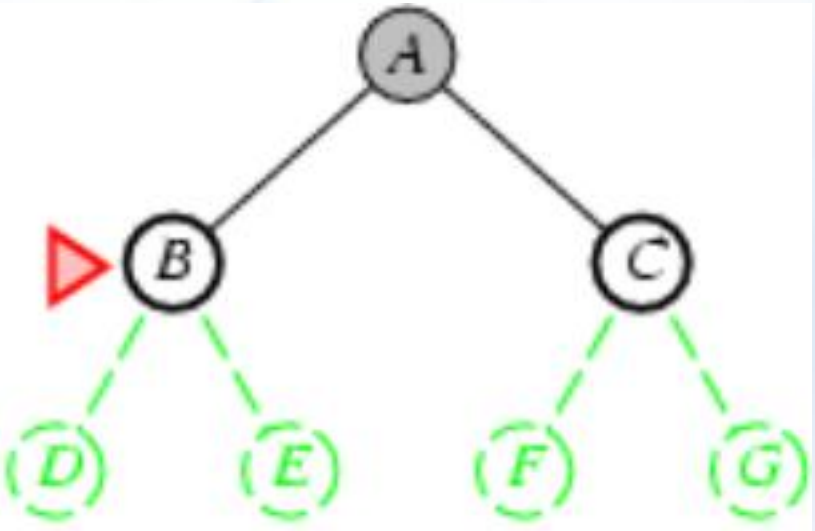
- **Implementation:**
  - *fringe* is a FIFO queue, i.e., new successors go at end



Breadth-first search

- **Expand shallowest unexpanded node**

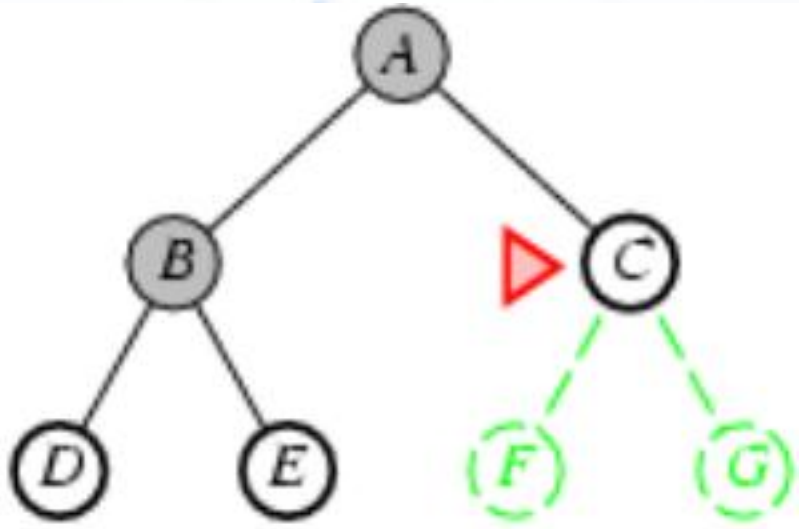
- **Implementation:**
  - *fringe* is a FIFO queue, i.e., new successors go at end



Breadth-first search

- **Expand shallowest unexpanded node**

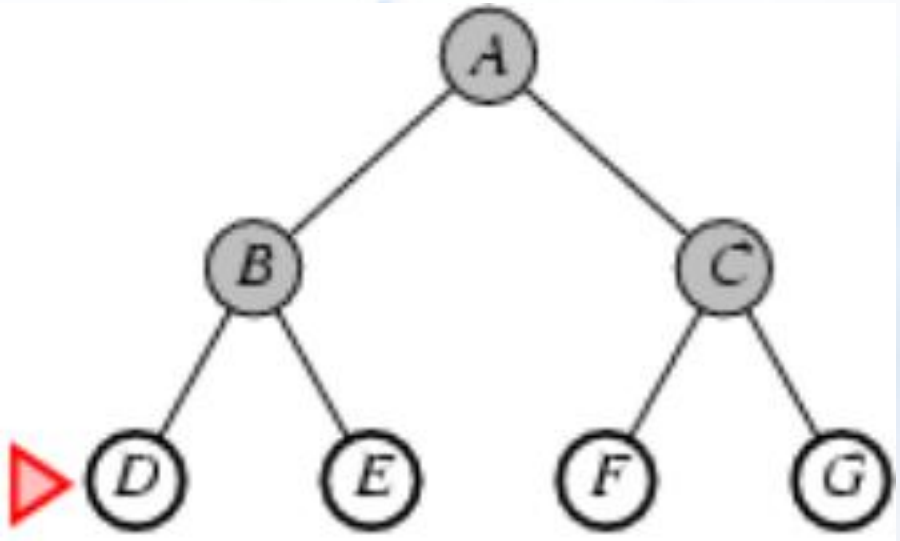
- **Implementation:**
  - *fringe* is a FIFO queue, i.e., new successors go at end



Breadth-first search

- **Expand shallowest unexpanded node**

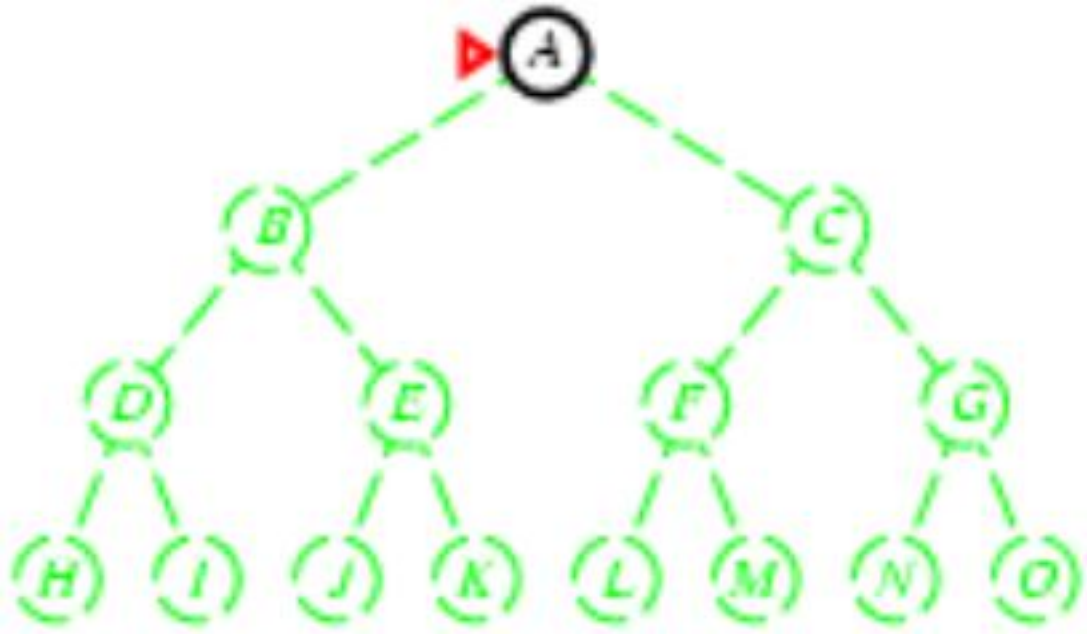
- **Implementation:**
  - *fringe* is a FIFO queue, i.e., new successors go at end



(2) Depth-first search

- **Expand deepest unexpanded node**

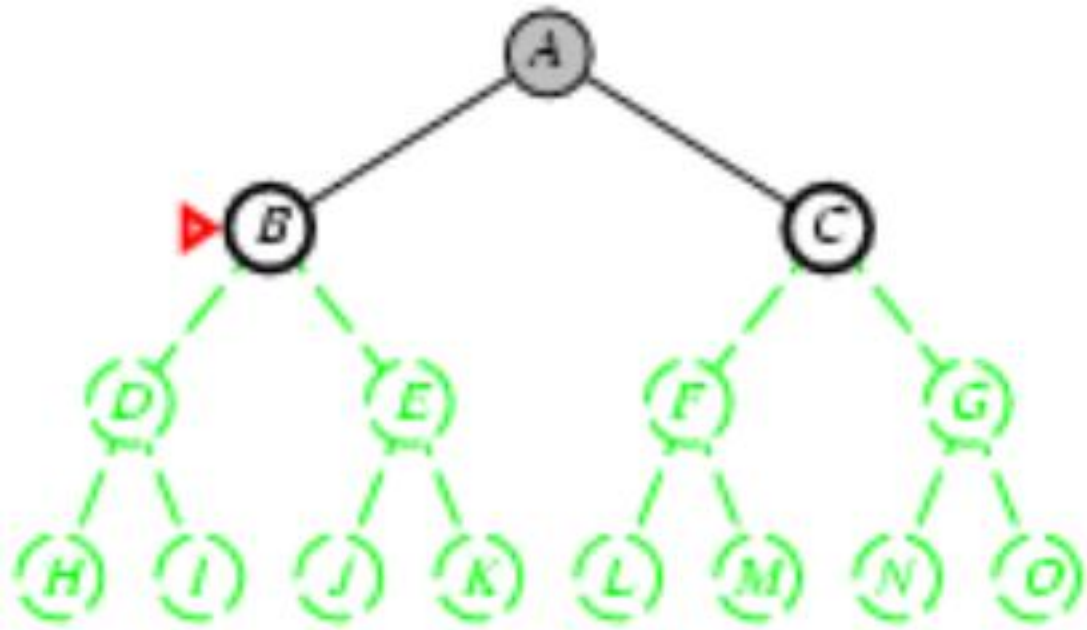
- **Implementation:**
  - *fringe* = LIFO queue, i.e., put successors at front



Depth-first search

- **Expand deepest unexpanded node**

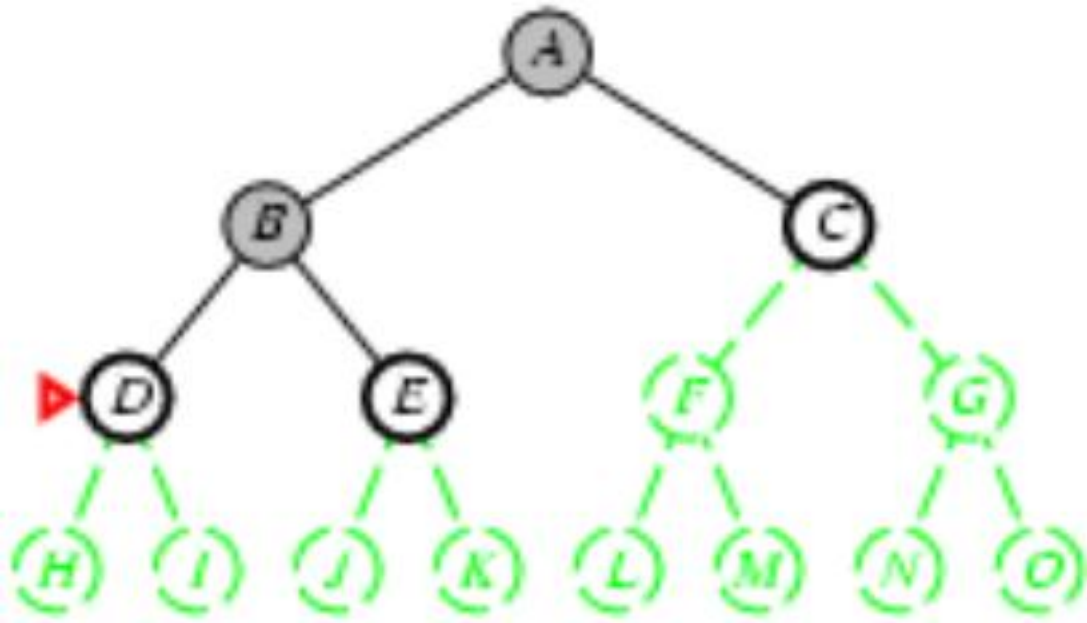
- **Implementation:**
  - *fringe* = LIFO queue, i.e., put successors at front



Depth-first search

- **Expand deepest unexpanded node**

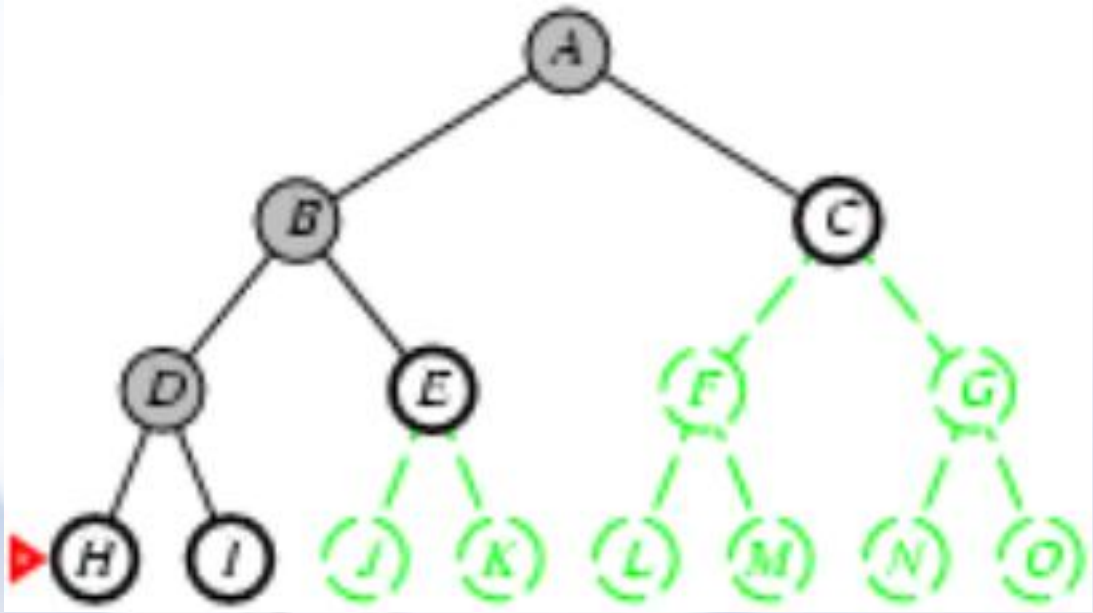
- **Implementation:**
  - *fringe* = LIFO queue, i.e., put successors at front



Depth-first search

- **Expand deepest unexpanded node**

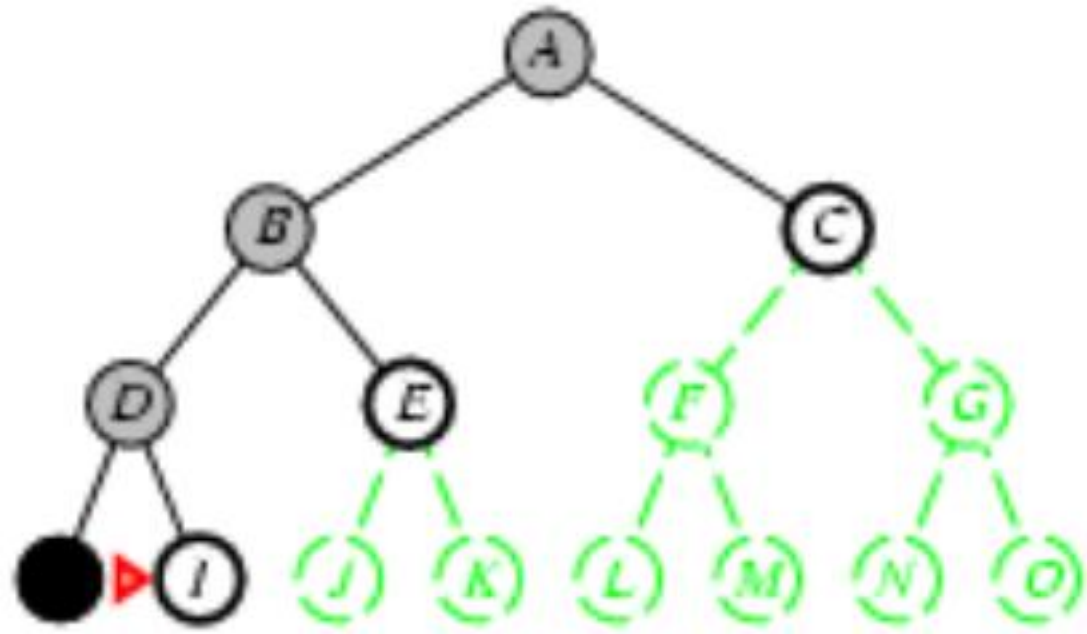
- **Implementation:**
  - *fringe* = LIFO queue, i.e., put successors at front



Depth-first search

- **Expand deepest unexpanded node**

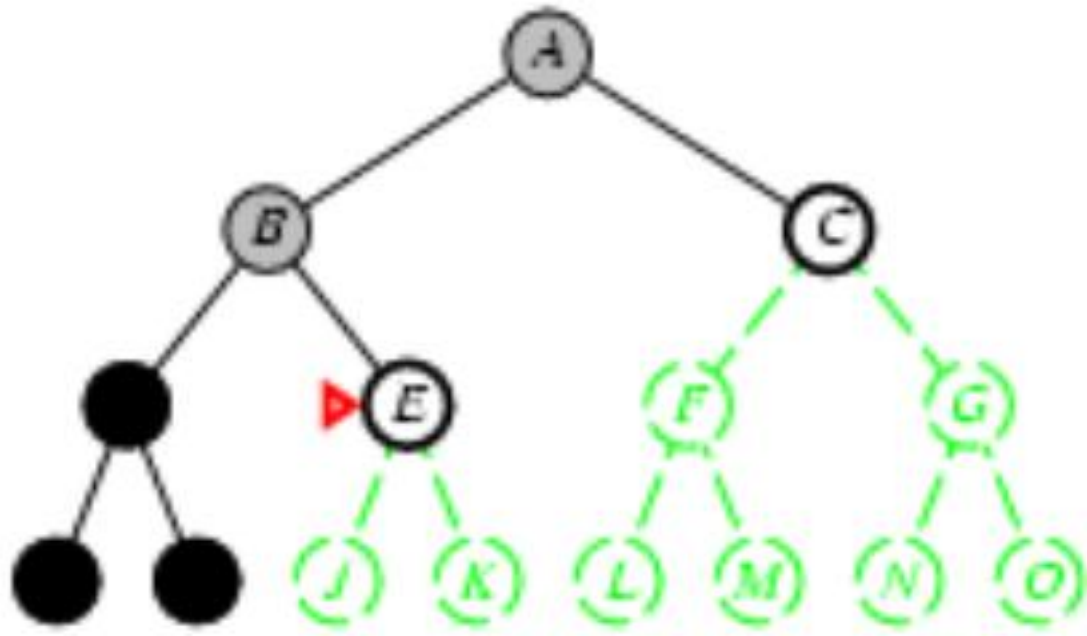
- **Implementation:**
  - *fringe* = LIFO queue, i.e., put successors at front



Depth-first search

- **Expand deepest unexpanded node**

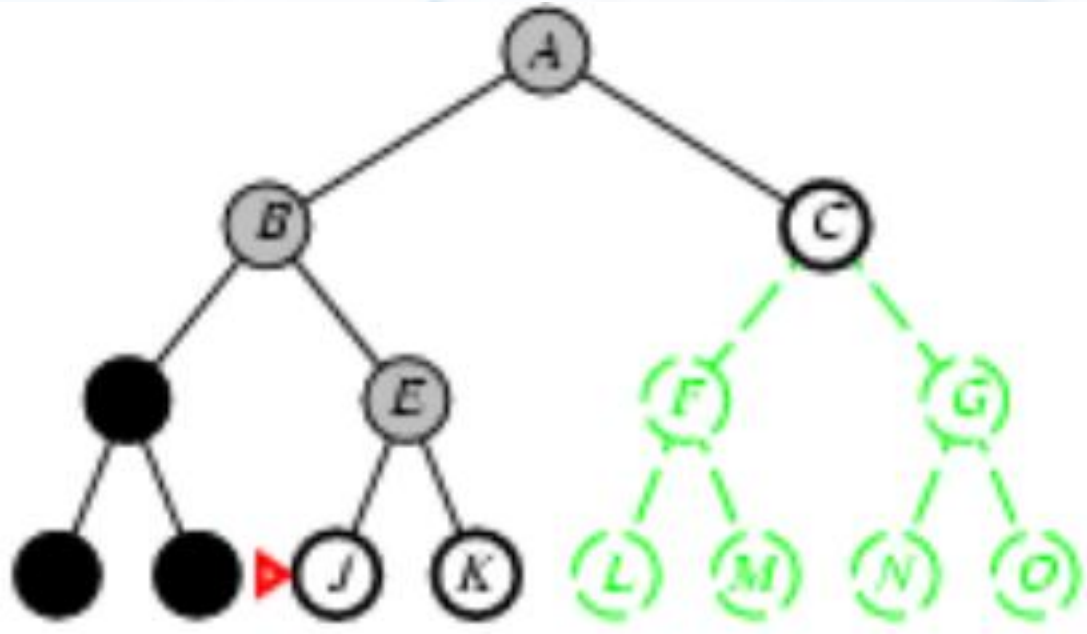
- **Implementation:**
  - *fringe* = LIFO queue, i.e., put successors at front



Depth-first search

- **Expand deepest unexpanded node**

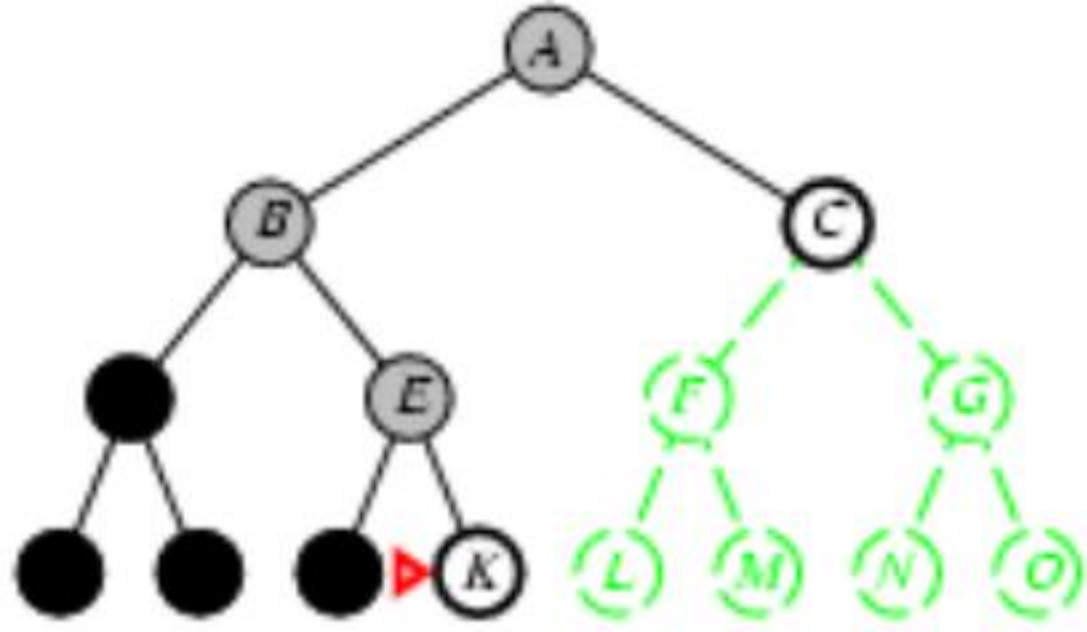
- **Implementation:**
  - *fringe* = LIFO queue, i.e., put successors at front



Depth-first search

- **Expand deepest unexpanded node**

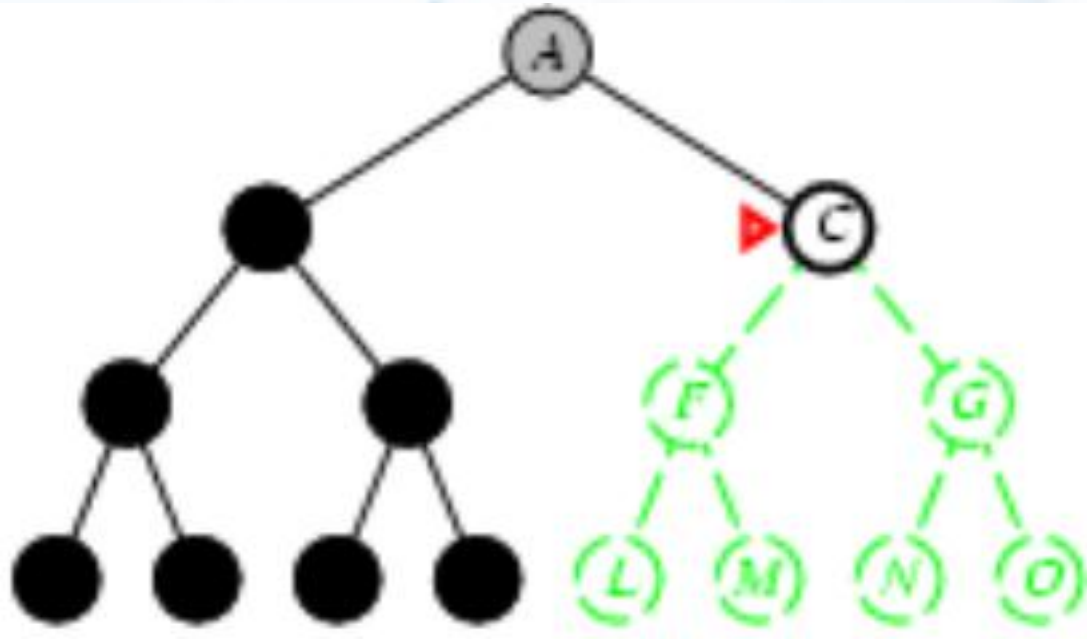
- **Implementation:**
  - *fringe* = LIFO queue, i.e., put successors at front



Depth-first search

- **Expand deepest unexpanded node**

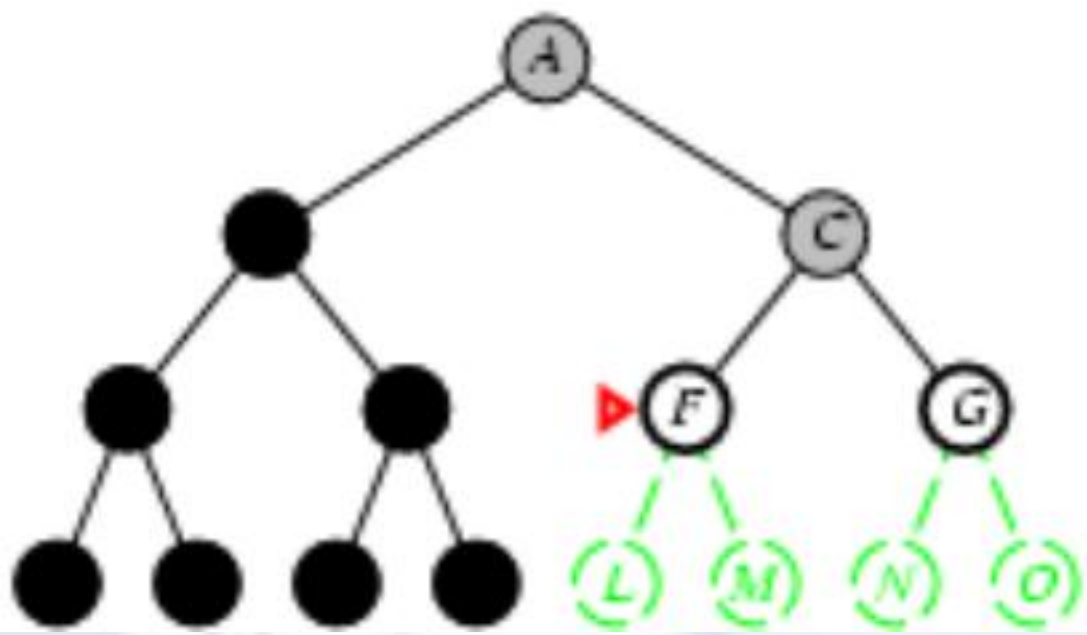
- **Implementation:**
  - *fringe* = LIFO queue, i.e., put successors at front



Depth-first search

- **Expand deepest unexpanded node**

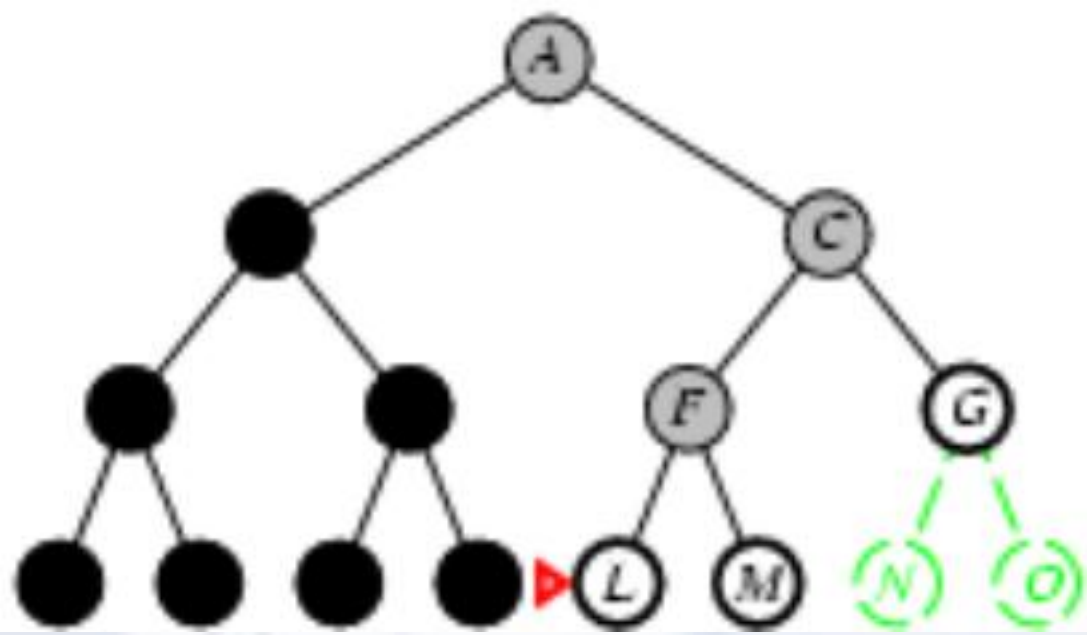
- **Implementation:**
  - *fringe* = LIFO queue, i.e., put successors at front



Depth-first search

- **Expand deepest unexpanded node**

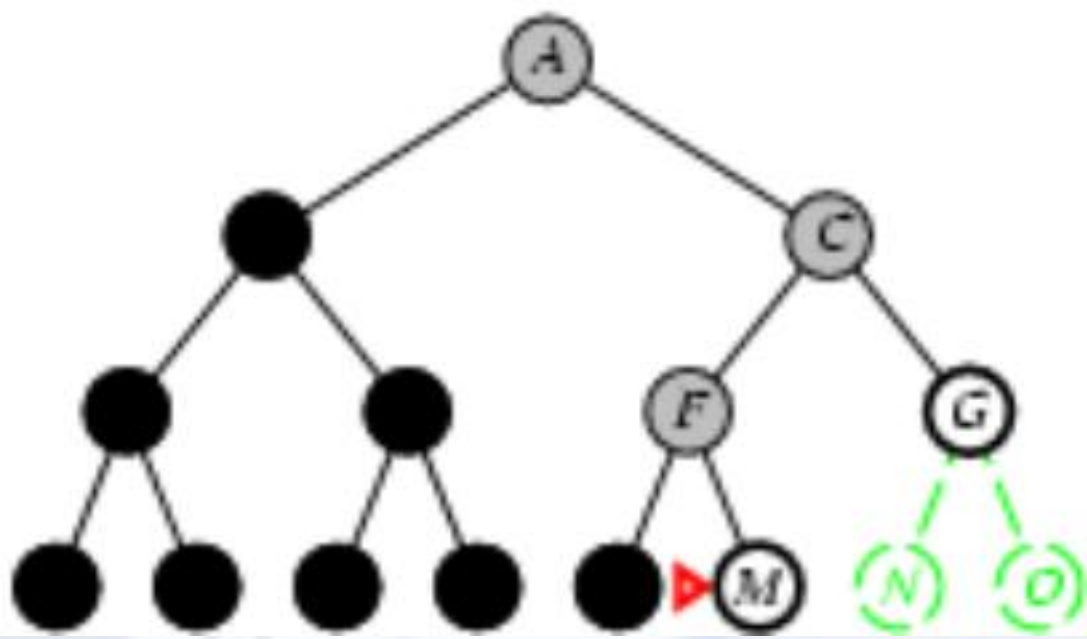
- **Implementation:**
  - *fringe* = LIFO queue, i.e., put successors at front



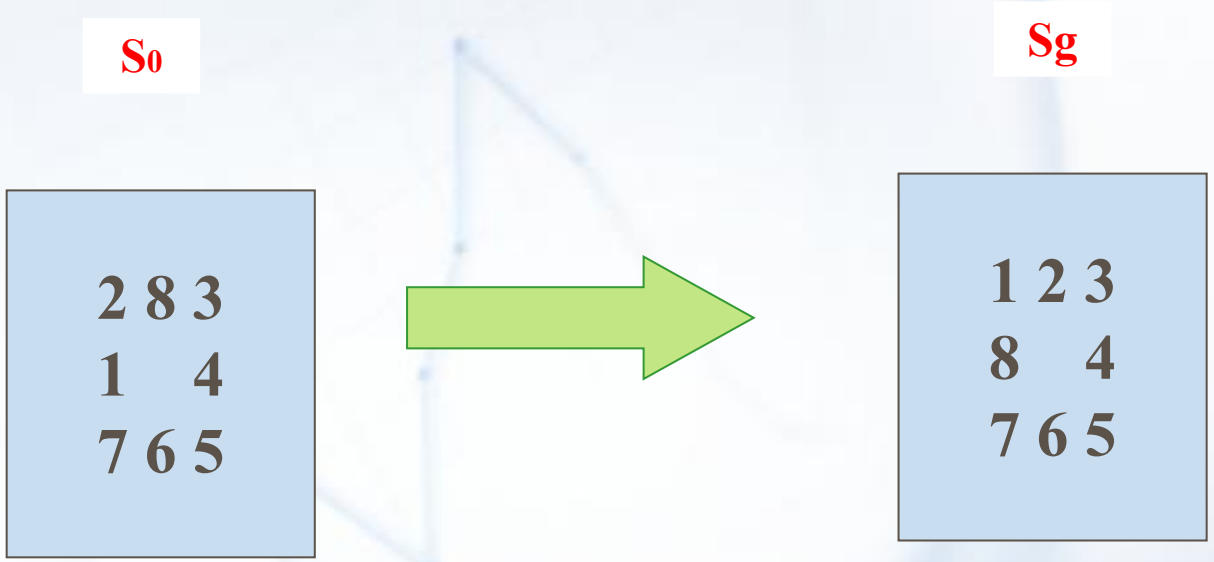
Depth-first search

- **Expand deepest unexpanded node**

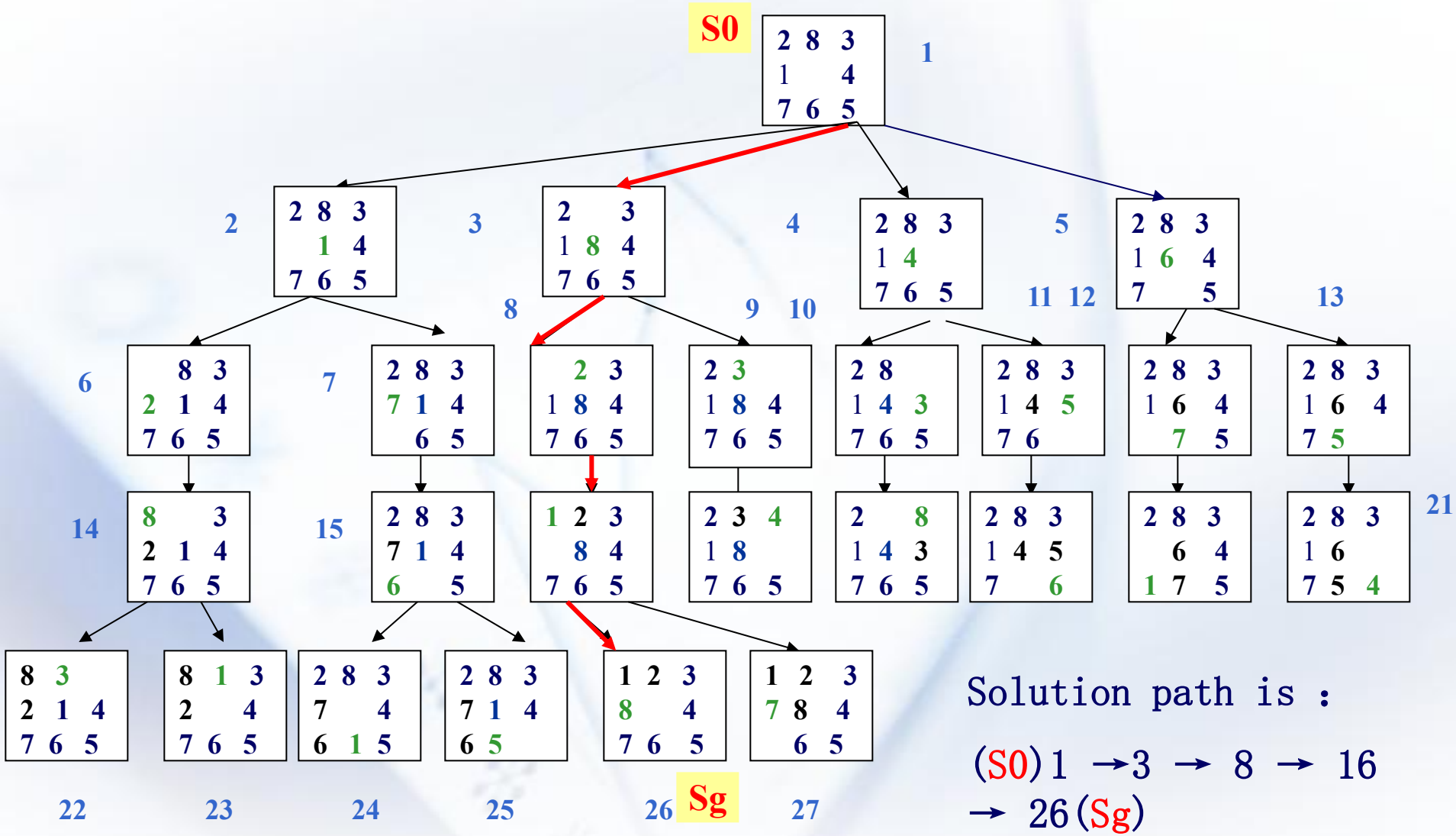
- **Implementation:**
  - *fringe* = LIFO queue, i.e., put successors at front



Blind Search Example: 8-puzzle



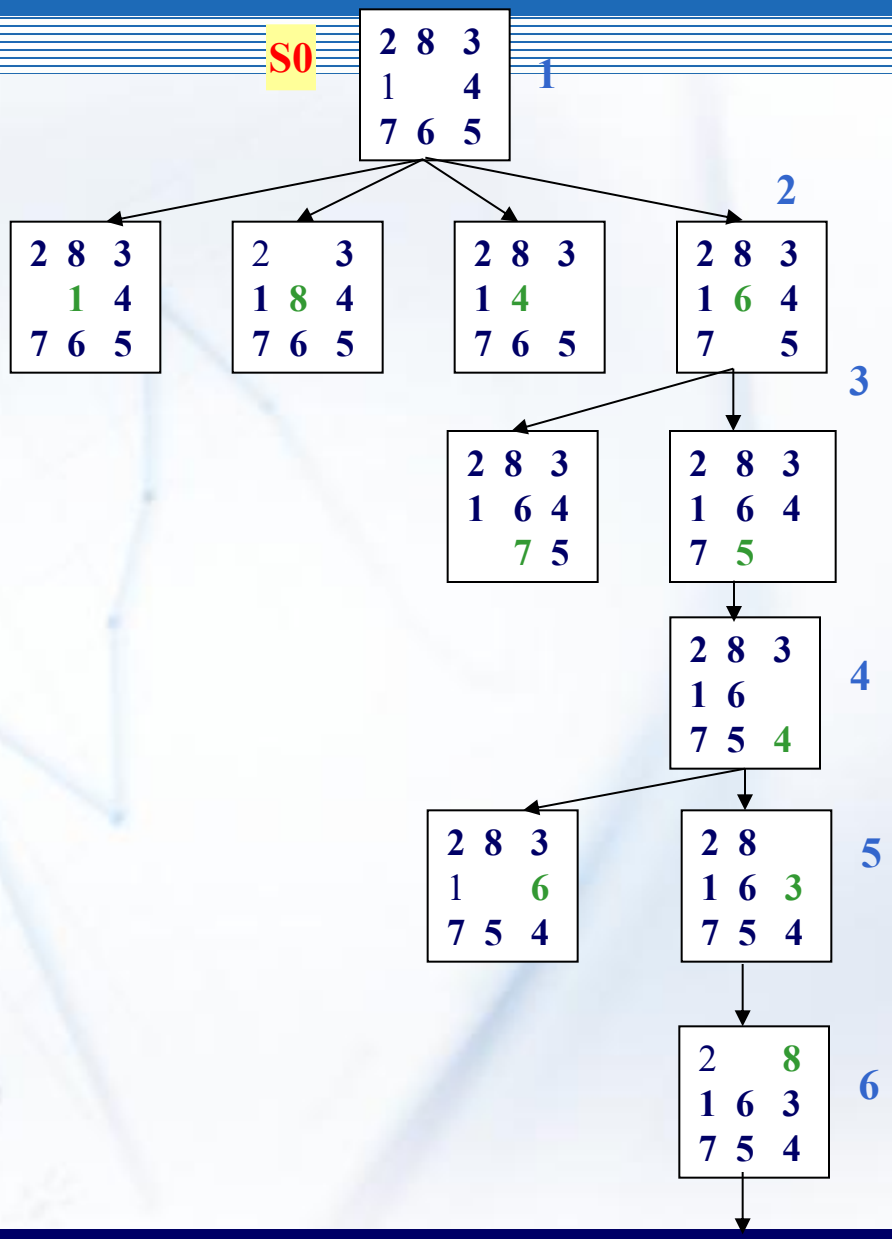
BFS for 8-puzzle



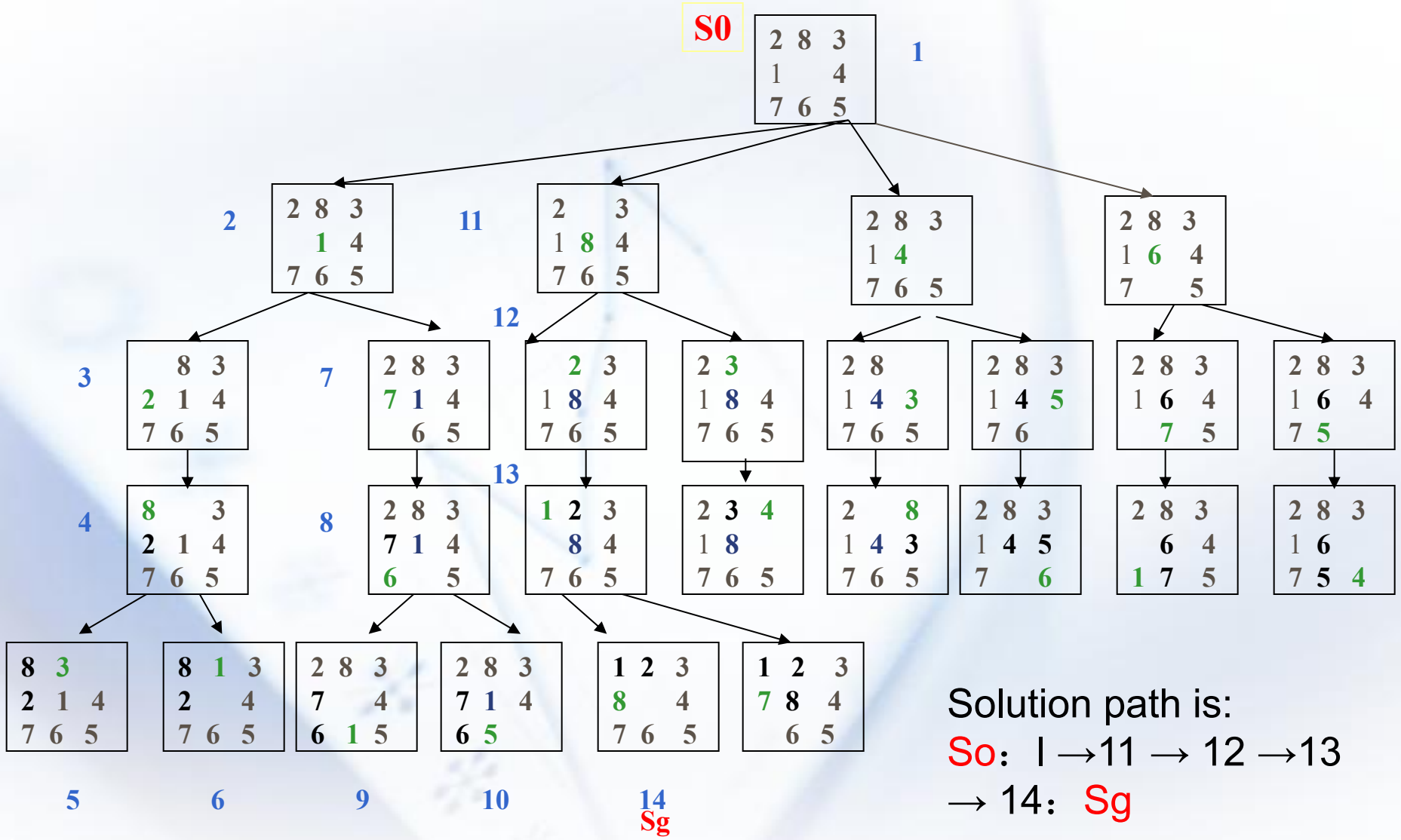
Solution path is :

(S0) 1 → 3 → 8 → 16 → 26 (Sg)

DFS for 8-puzzle



DLS for 8-puzzle (depth limitation=4)



Solution path is:

So: I →11 → 12 →13 → 14: Sg

3.3 Heuristic Search

- **Review:** The key difference between different search strategies is in the order of node expansion.
- **Idea:** use an evaluation function *f(n)* to Estimate "desirability" of each node, and Expand most desirable unexpanded node.

- **Special cases:**
  - A* algorithm
  - AO* algorithm

（1）A* algorithm

- for State Space search
- Evaluation function

$$f(n) = g(n) + h(n)$$

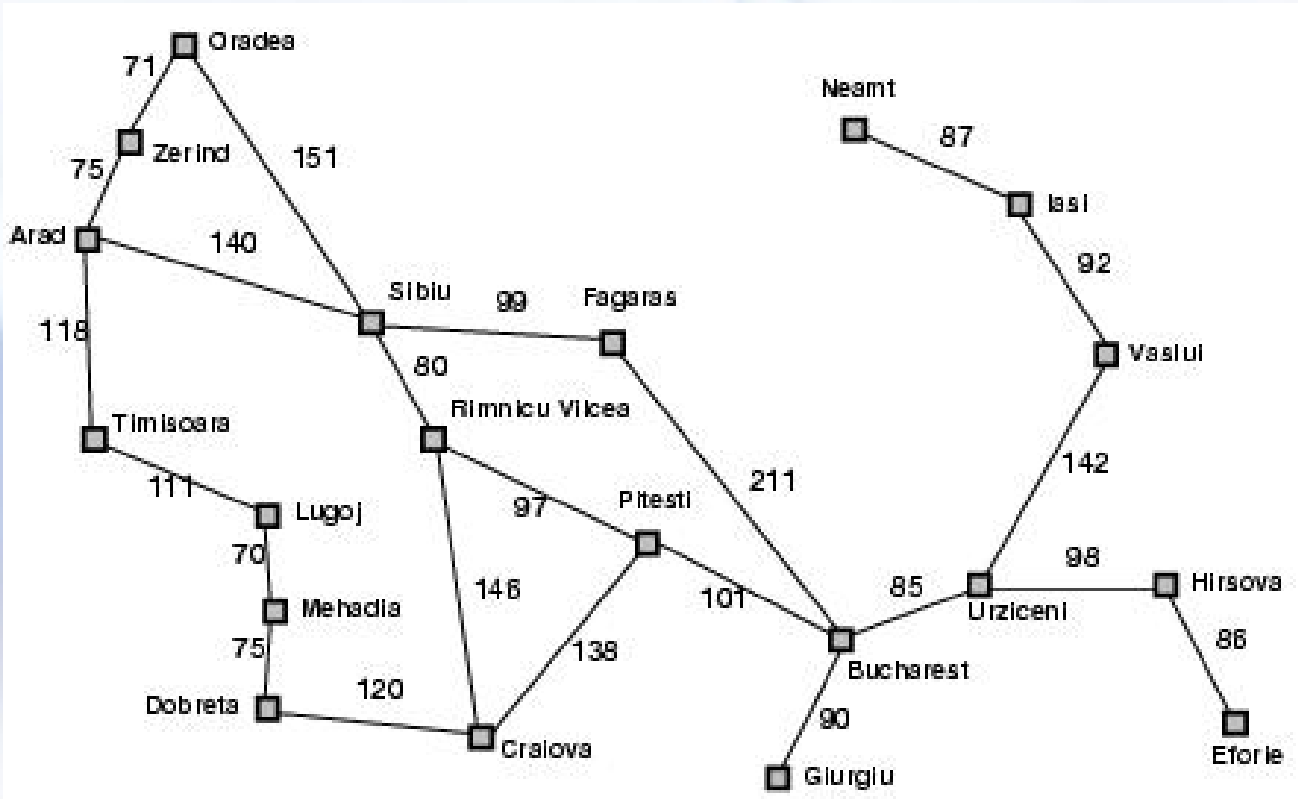
$g(n)$ = cost so far to reach n

$h(n)$ = estimated cost from n to goal (heuristic function)

$f(n)$ = estimated total cost of path through n to goal

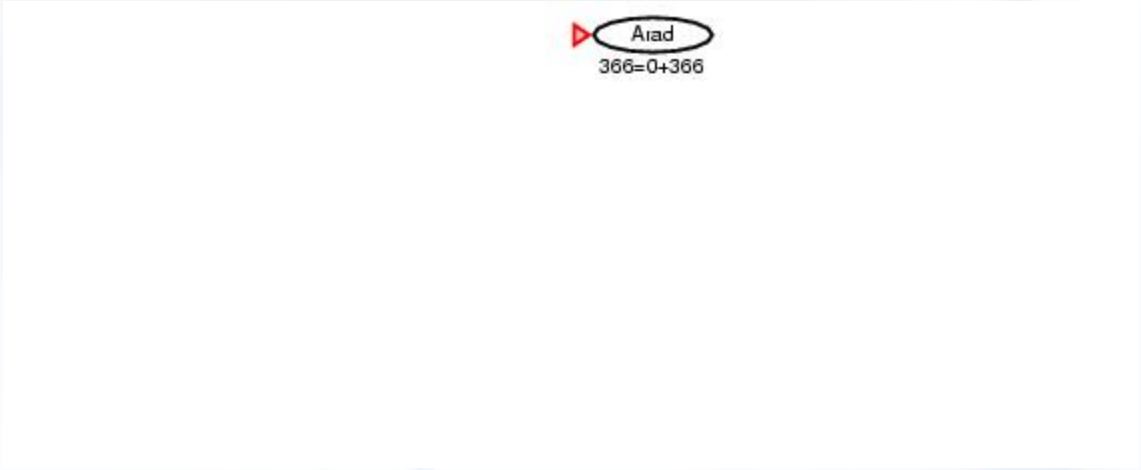
Example: Romania Problem

To find out an optimal path from the start city (e.g. Arad) to the destination (e.g. Bucharest) in the following map.



Straight–line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	10
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

A* search for Romania problem



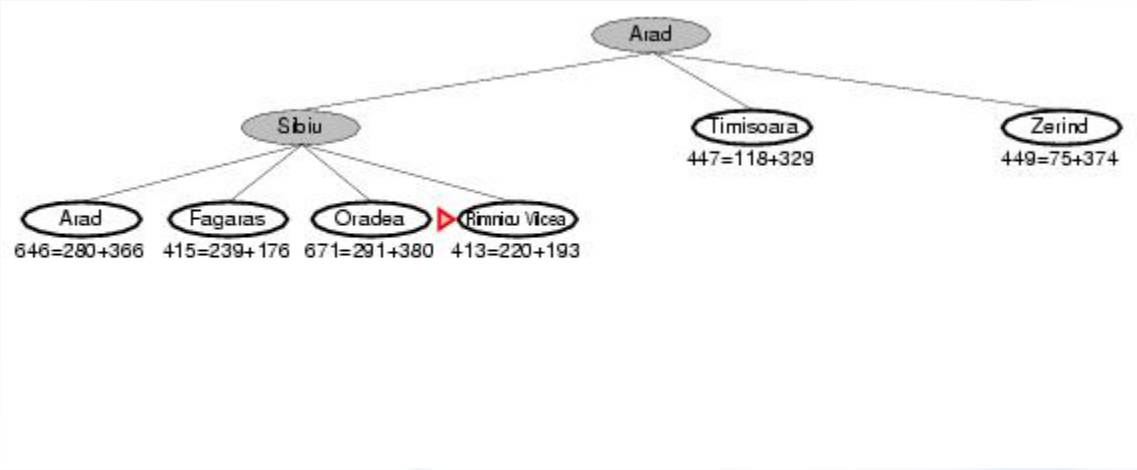
Use the straight-line distance between two cities as the heuristic function ($h_{SLD}(n)$)

A* search for Romania problem



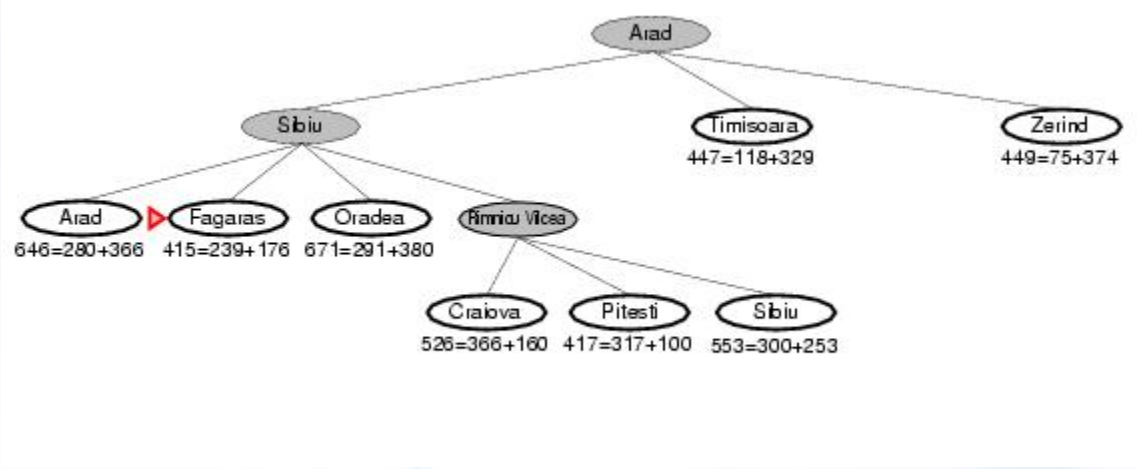
Use the straight-line distance between two cities as the heuristic function ($h_{SLD}(n)$)

A* search for Romania problem



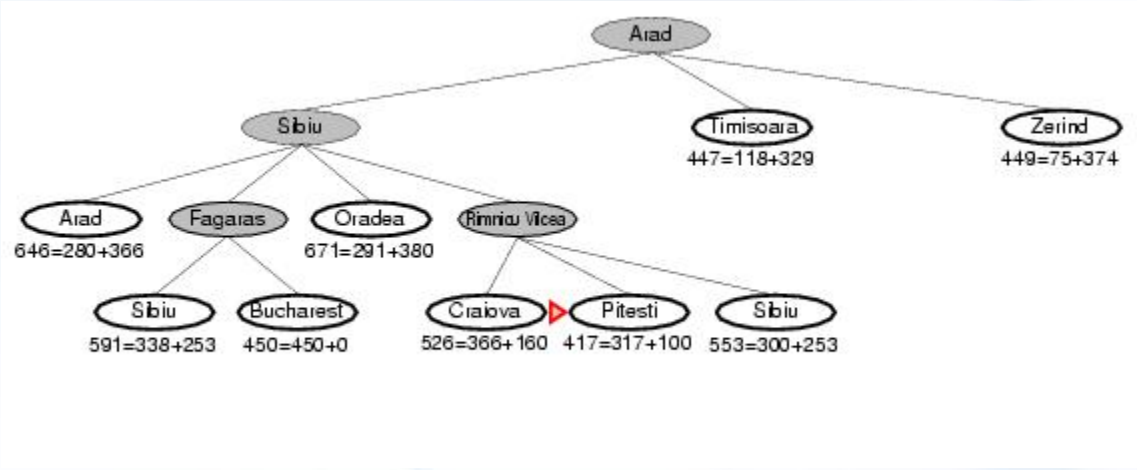
Use the straight-line distance between two cities as the heuristic function ($h_{SLD}(n)$)

A* search for Romania problem



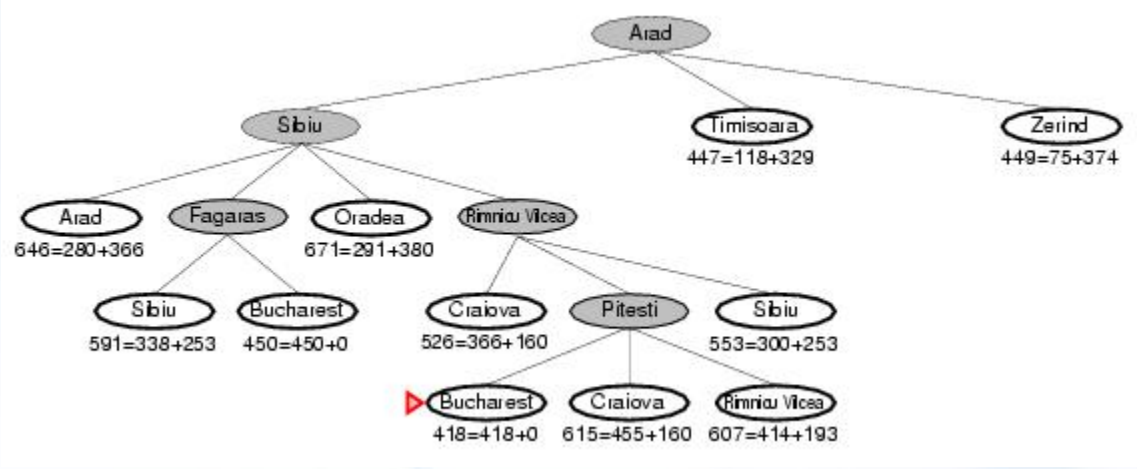
Use the straight-line distance between two cities as the heuristic function ($h_{SLD}(n)$)

A* search for Romania problem



Use the straight-line distance between two cities as the heuristic function ($h_{SLD}(n)$)

A* search for Romania problem



Use the straight-line distance between two cities as the heuristic function ($h_{SLD}(n)$)

Admissible heuristics

- A heuristic *h(n)* is admissible if for every node *n*, $h(n) \leq h^*(n)$, where $h^*(n)$ is the true cost to reach the goal state from *n*.

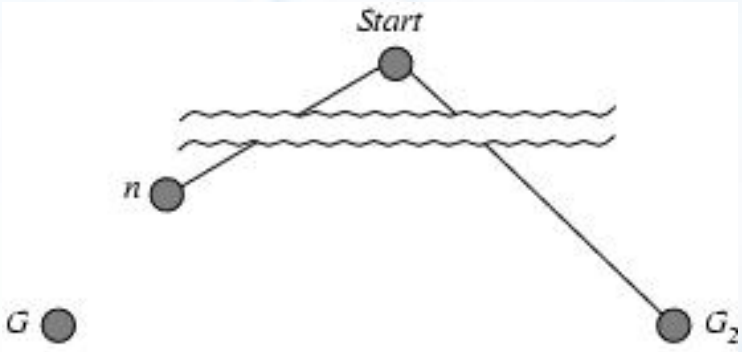
- An admissible heuristic never overestimates the cost to reach the goal, i.e., it is optimistic

  Example: $h_{SLD}(n)$ never overestimates the actual road distance.

- Theorem: If *h(n)* is admissible, A* is optimal for tree search

Optimality of A* (proof)

- **Suppose some suboptimal goal G_2 has been generated and is in the fringe. Let *n* be an unexpanded node in the fringe such that *n* is on a shortest path to an optimal goal *G*.**



$f(G_2) = g(G_2)$ since $h(G_2) = 0$

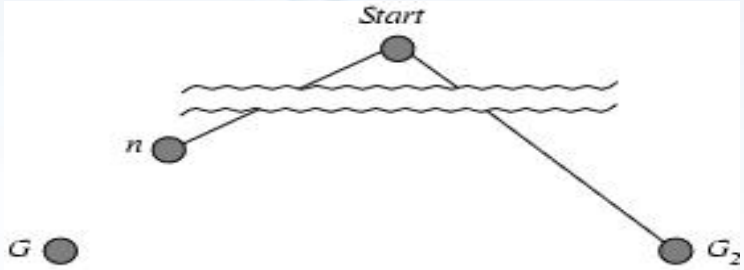
$g(G_2) > g(G)$ since G_2 is suboptimal

$f(G) = g(G)$ since $h(G) = 0$

$f(G_2) > f(G)$ from above

Optimality of A* (proof)

- Suppose some suboptimal goal G_2 has been generated and is in the fringe. Let *n* be an unexpanded node in the fringe such that *n* is on a shortest path to an optimal goal *G*.

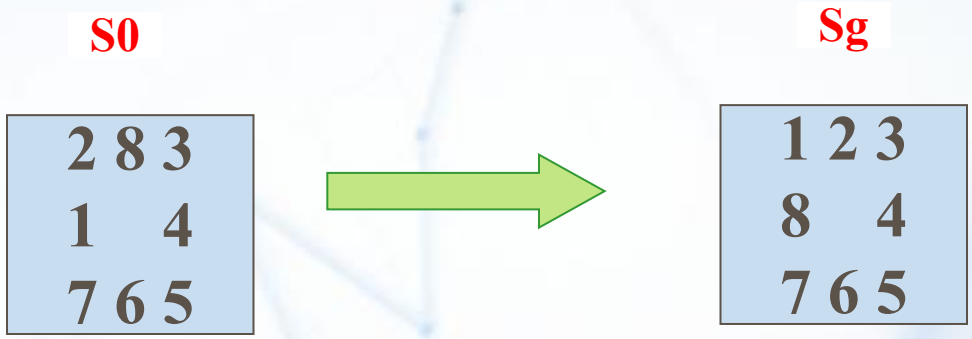


$f(G_2)$	$> f(G)$	from above
$h(n)$	$\leq h^*(n)$	since h is admissible
$g(n) + h(n)$	$\leq g(n) + h^*(n)$	
$f(n)$	$\leq f(G)$	

Hence $f(G_2) > f(n)$, and A^* will never select G_2 for expansion

A* Example: 8-puzzle

- $h_1(n)$ = number of misplaced tiles
- $h_2(n)$ = total Manhattan distance
  (i.e., no. of squares from desired location of each tile)



- $h_1(S)$ = ?
- $h_2(S)$ = ?

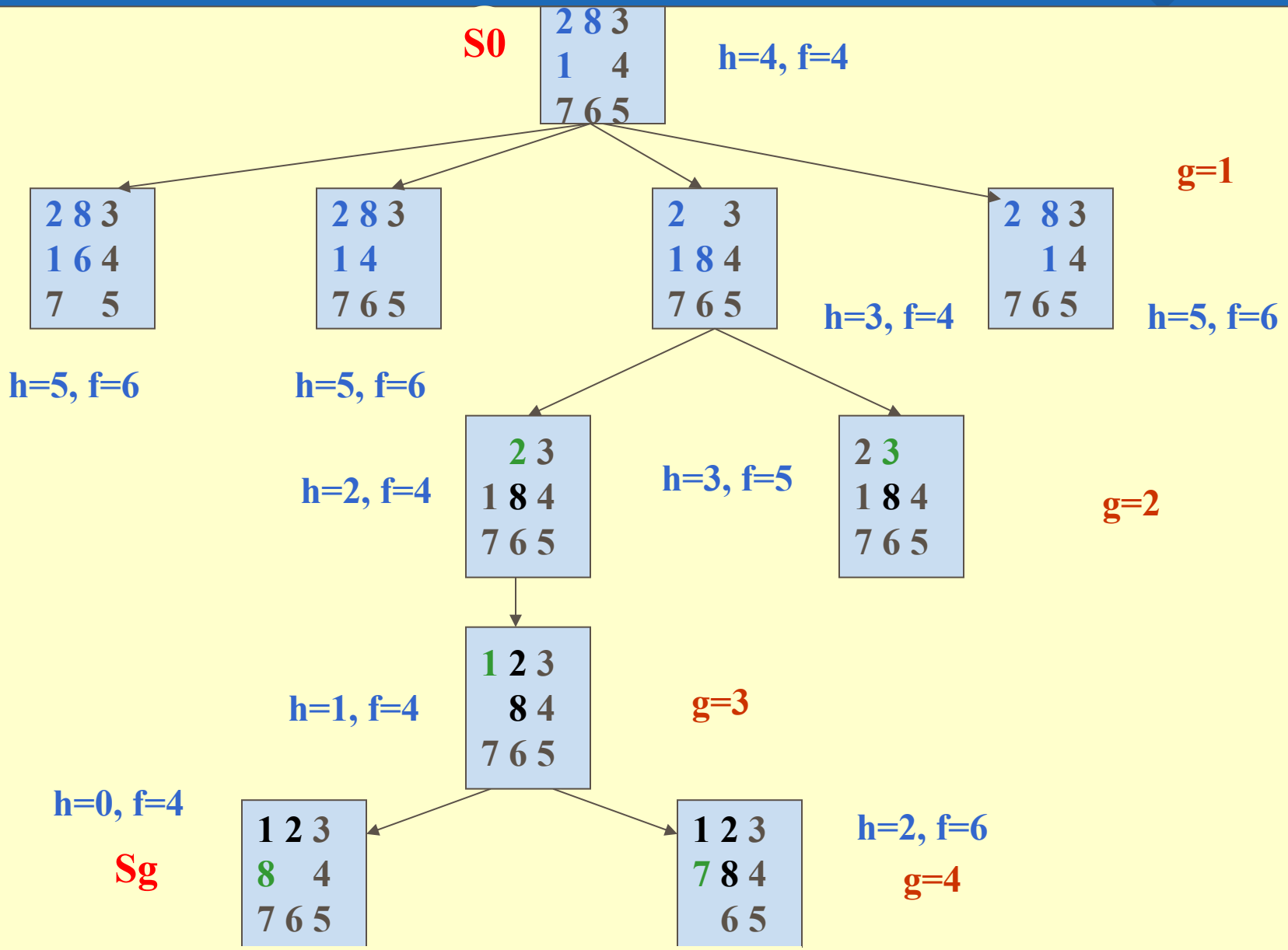
A* Example: 8-puzzle

- $h_1(n)$ = number of misplaced tiles
- $h_2(n)$ = total Manhattan distance
  (i.e., no. of squares from desired location of each tile)



- $h_1(S) = ?$ 3
- $h_2(S) = ?$ 1+1+2 = 4

A* for 8-puzzle (using $h_2(n)$)



Design heuristic functions: Dominance

- If $h_2(n) \geq h_1(n)$ for all n (both admissible), then h_2 dominates h_1, and h_2 is better for search

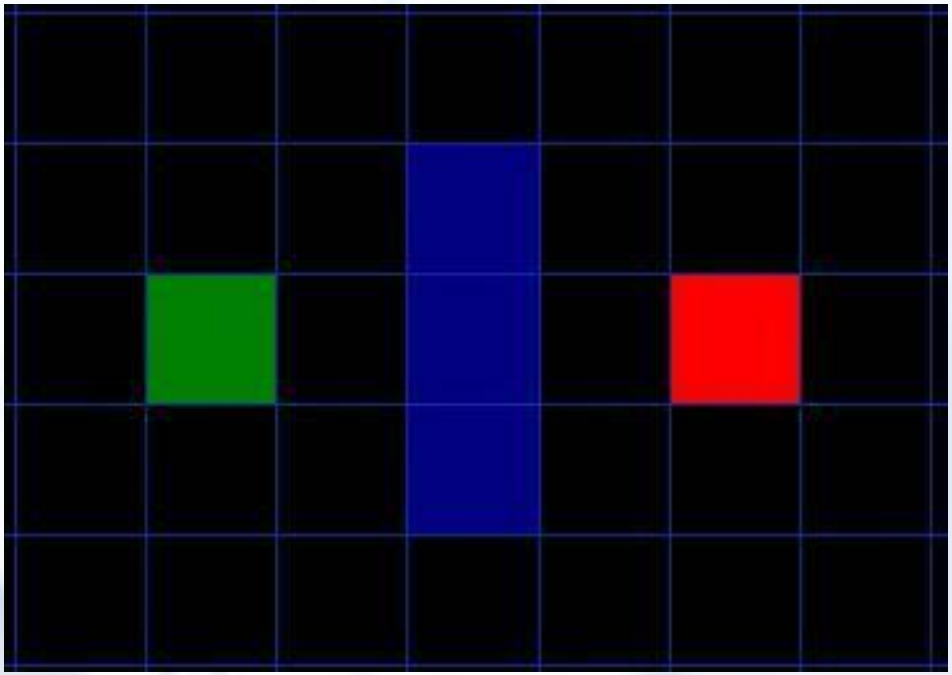
- Typical search costs (average number of nodes expanded):

  $d=12$ IDS = 3,644,035 nodes
  $A^*(h_1)$ = 227 nodes
  $A^*(h_2)$ = 73 nodes

  $d=24$ IDS = too many nodes
  $A^*(h_1)$ = 39,135 nodes
  $A^*(h_2)$ = 1,641 nodes

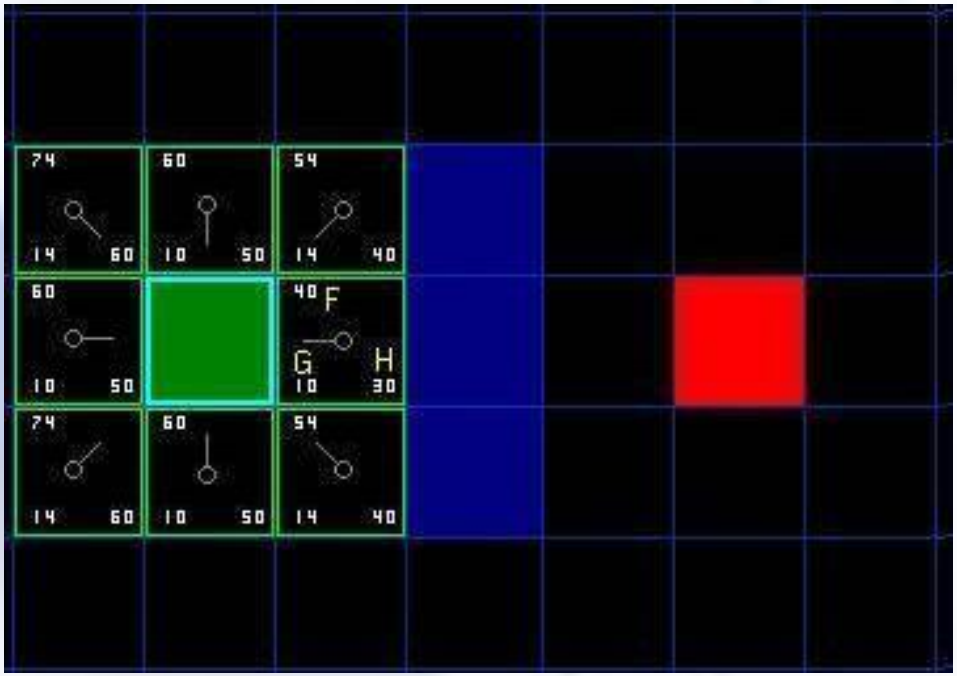
A* example: path finding in game Problem

- How to get from point A to point B, which are separated by a wall ?



A* example: path finding in game Problem

- **Starting at point A, checking the adjacent squares and calculating the heuristic value F (F = G + H) for each square.**

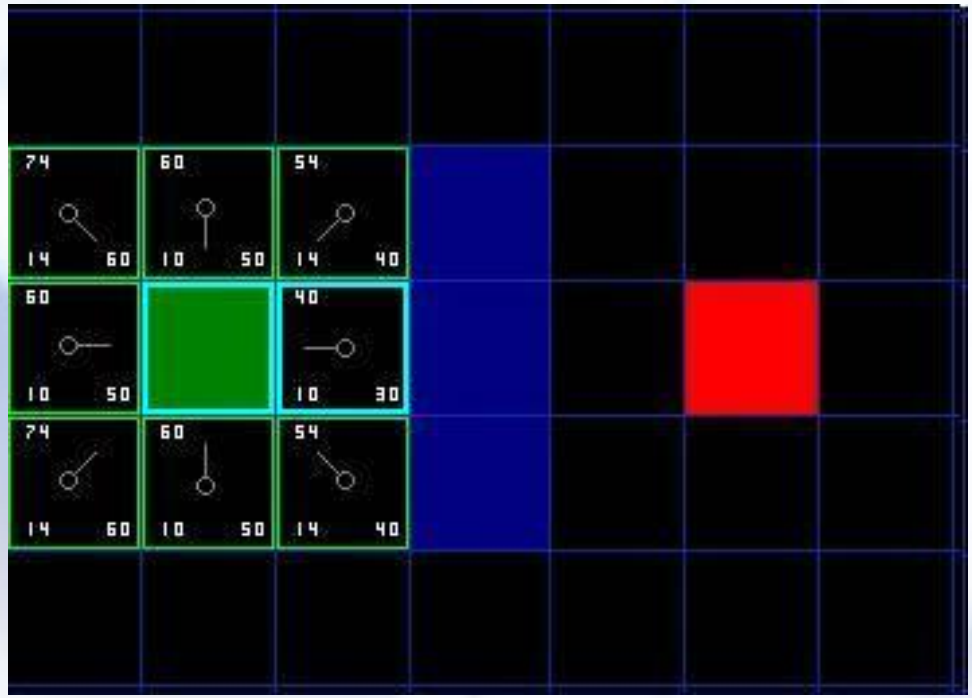


H: the total number of squares moved horizontally and vertically to reach the target square from the current square

G: the movement cost to move from the starting point to the given square: A cost of 10 to each horizontal or vertical square moved, and a cost of 14 for a diagonal move.

A* example: path finding in game Problem

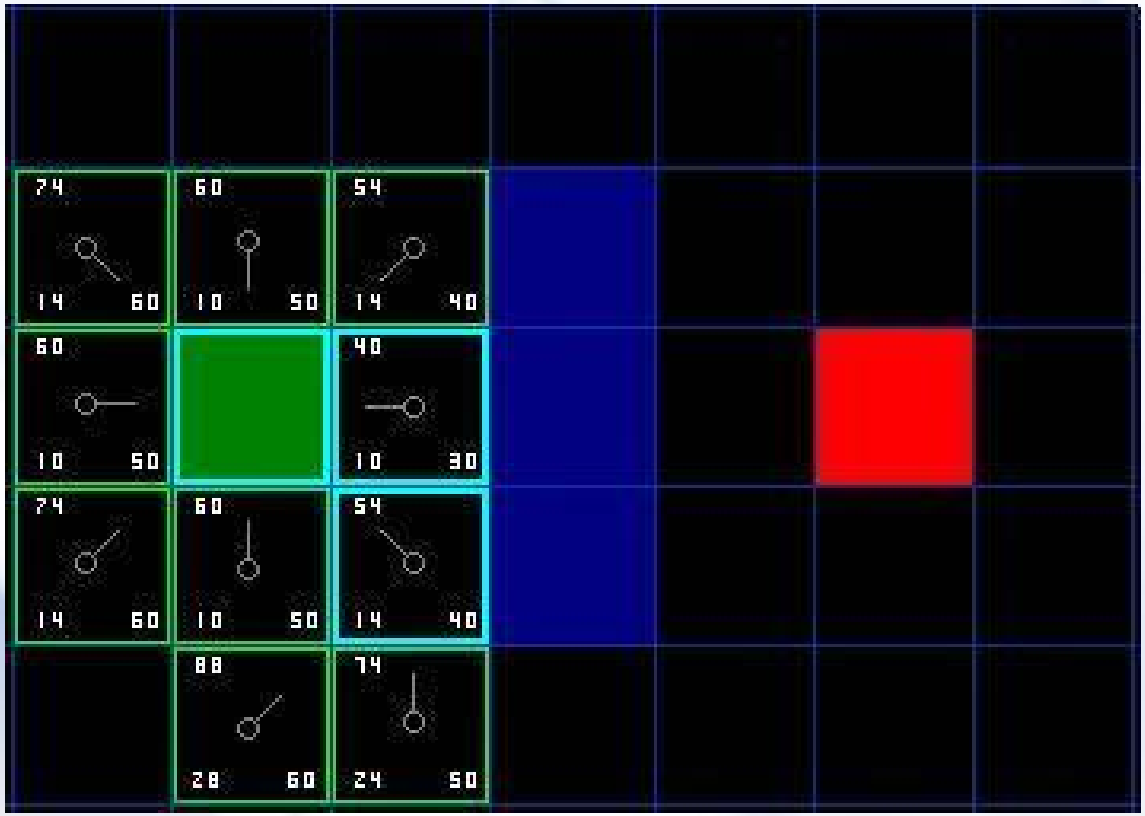
- **Determining the square with minimum heuristic value F (F = G + H) to follow.**



- choice is the immediate right of the starting square, with an F score of 40

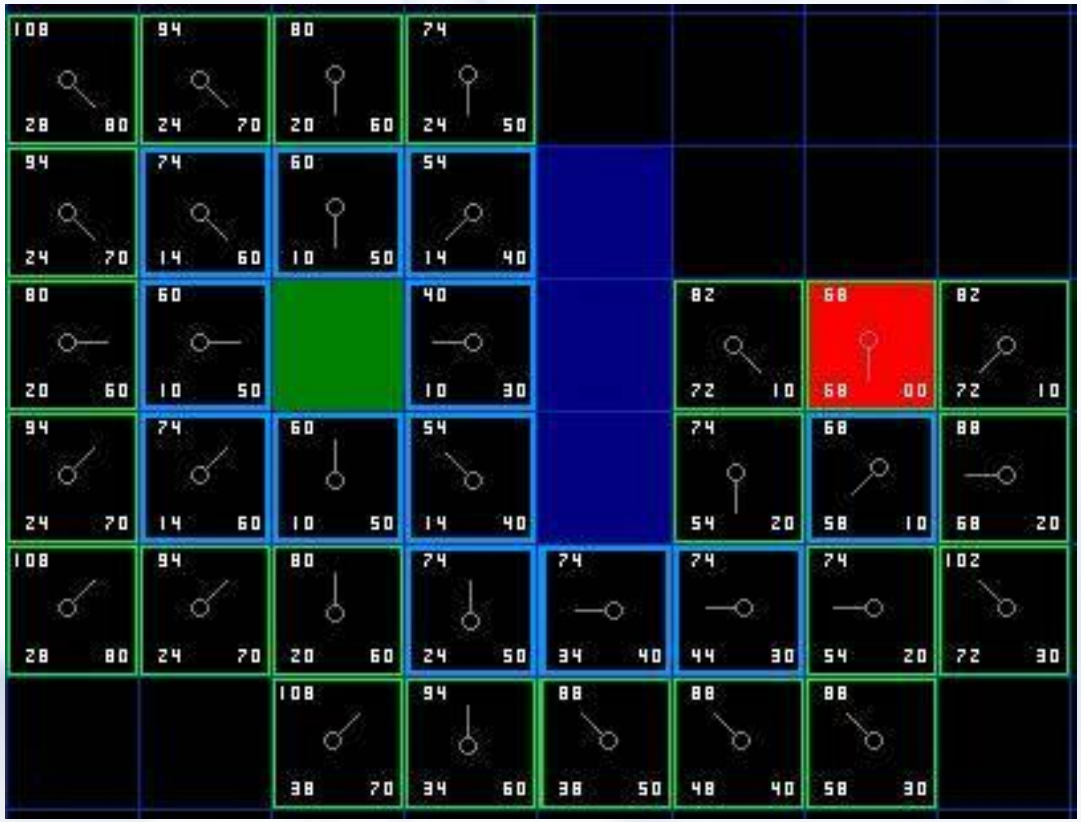
A* example: path finding in game Problem

- Continue the step2 for the current square, ignoring the squares blocked by the wall.



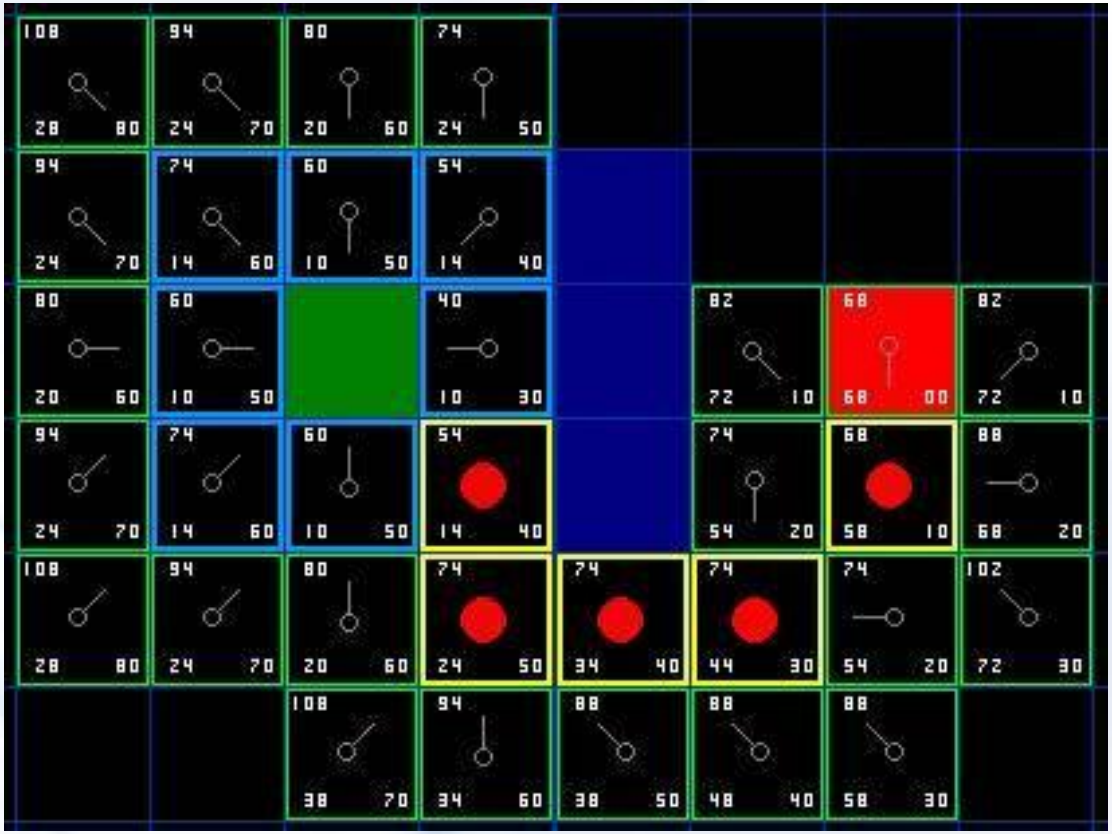
A* example: path finding in game Problem

- repeat this process until we **add the target square to the closed list**.



A* example: path finding in game Problem

- Determine the path: start at the red target square, and work backwards moving from one square to its parent, following the arrows.



Summary of A* algorithm

Step1. Put the initial node into open table

Step2. Select a node from open table and put it into closed table according to heuristic function.

Step3. Expand the selected node. If the node can be expanded, put all the expanded nodes into open table.

Step4. Repeat Step2-3 until the goal node is selected in step2, or open table is empty.

Part Ⅳ: Adversarial Search

4.1 Game tree

- Data Structure representing game process
  - 2-player (MAX vs. MIN)
  - deterministic
  - turn-taking
  - zero-sum
  - perfect information

Game Tree Example: Coins-Partition

Game State： $(K_1, K_2, \ldots, Kn,$ **MIN or MAX**$)$

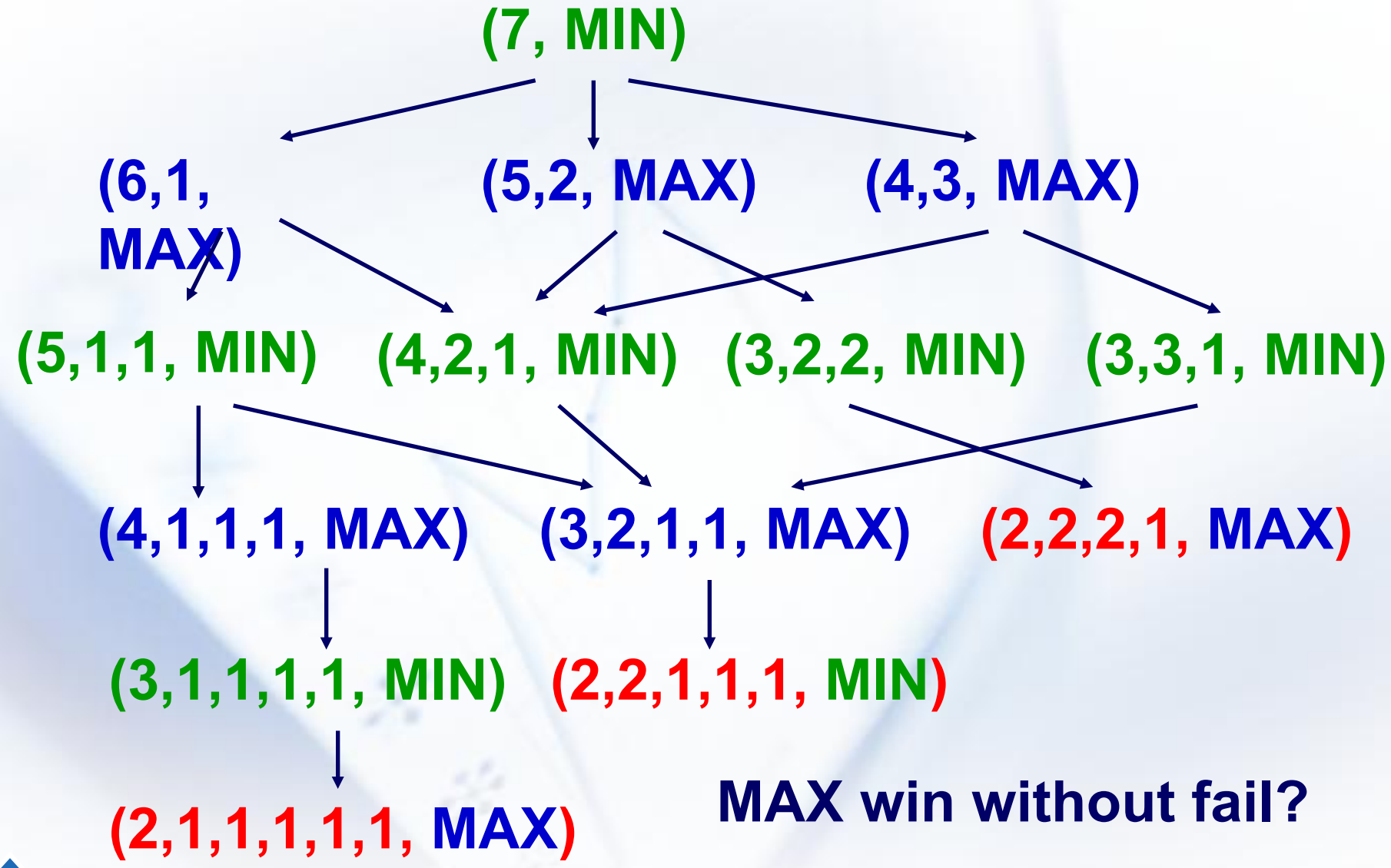
Numbers of coins in stacks

turn

Each time MIN or MAX choose one stack and divide it into two stacks with different numbers of coins. When MIN or MAX cannot finish this task, he lose.

Firstly, n=1, $k_1 = 7$, turn = MIN

Game tree for coins-partition



MAX win without fail?

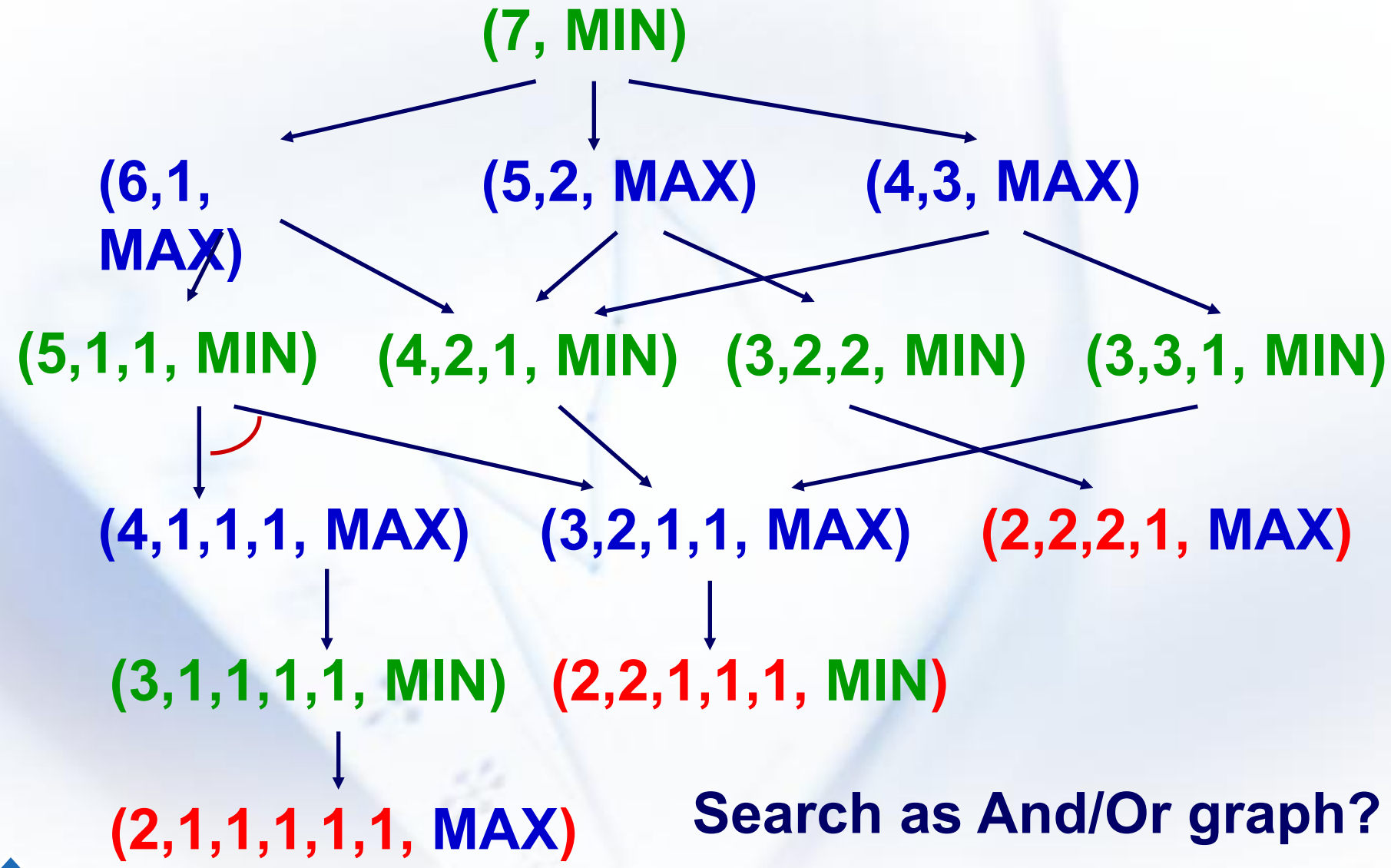
How to Solve?

- **MAX and MIN always try to take actions which are best for him in each step in the game.**
- **In order to win, MAX should choose the best action in each turn, at the same time he should consider that MIN also choose the best (oppositely, worst for MAX) action for him in each turn.**

best achievable payoff against best play

Game tree is And/Or Graph in structure!

Game tree as And/Or graph



Search as And/Or graph?

4.2 Minimax algorithm

- **Real Decision-Resource limits**

  E.g., China Chess: total number of states is 10^{160}. Suppose we explore 10^3 nodes/sec， we need 10^{145} years to find an optimal move.

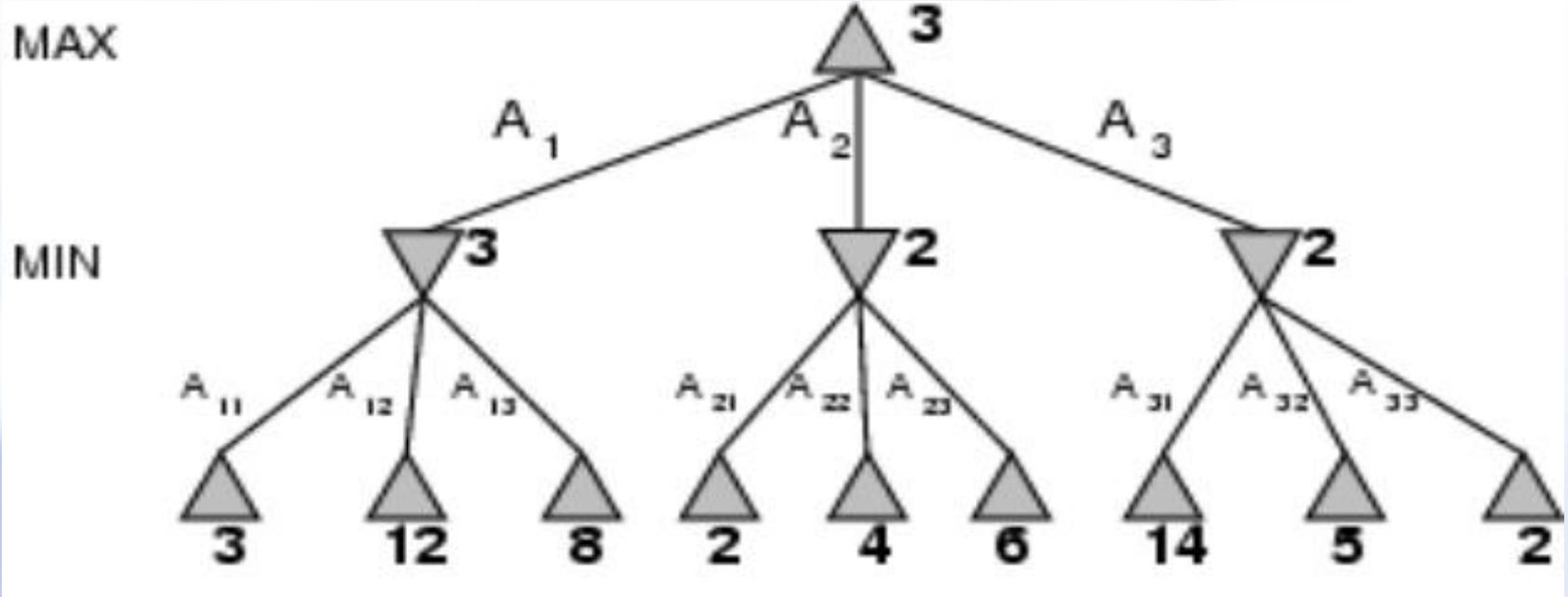
  (the age of universe is 10^{10})

❖**Solution**

Cutoff test: limit the depth of game tree

Evaluation function: Evaluate the game state from the view of MAX, the bigger the value is, the better the game state is.

Minimax Example：2-ply game

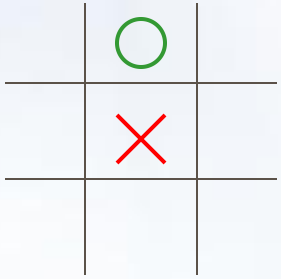


choose move to position with highest minmax value!

MiniMax Example: tic-tac-toe

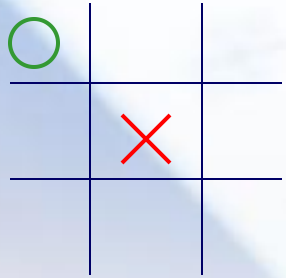
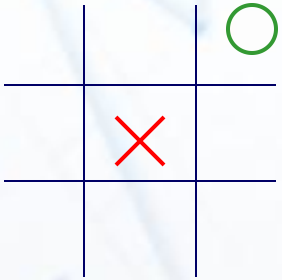
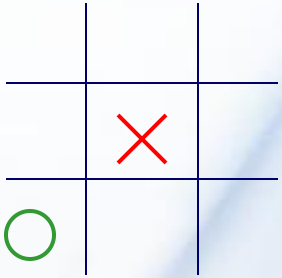
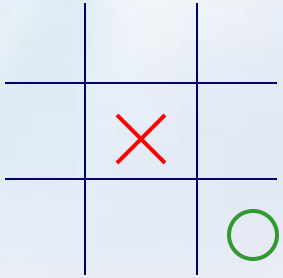
- **Evaluation function:**

$$e(P) = e(+P) - e(-P)$$

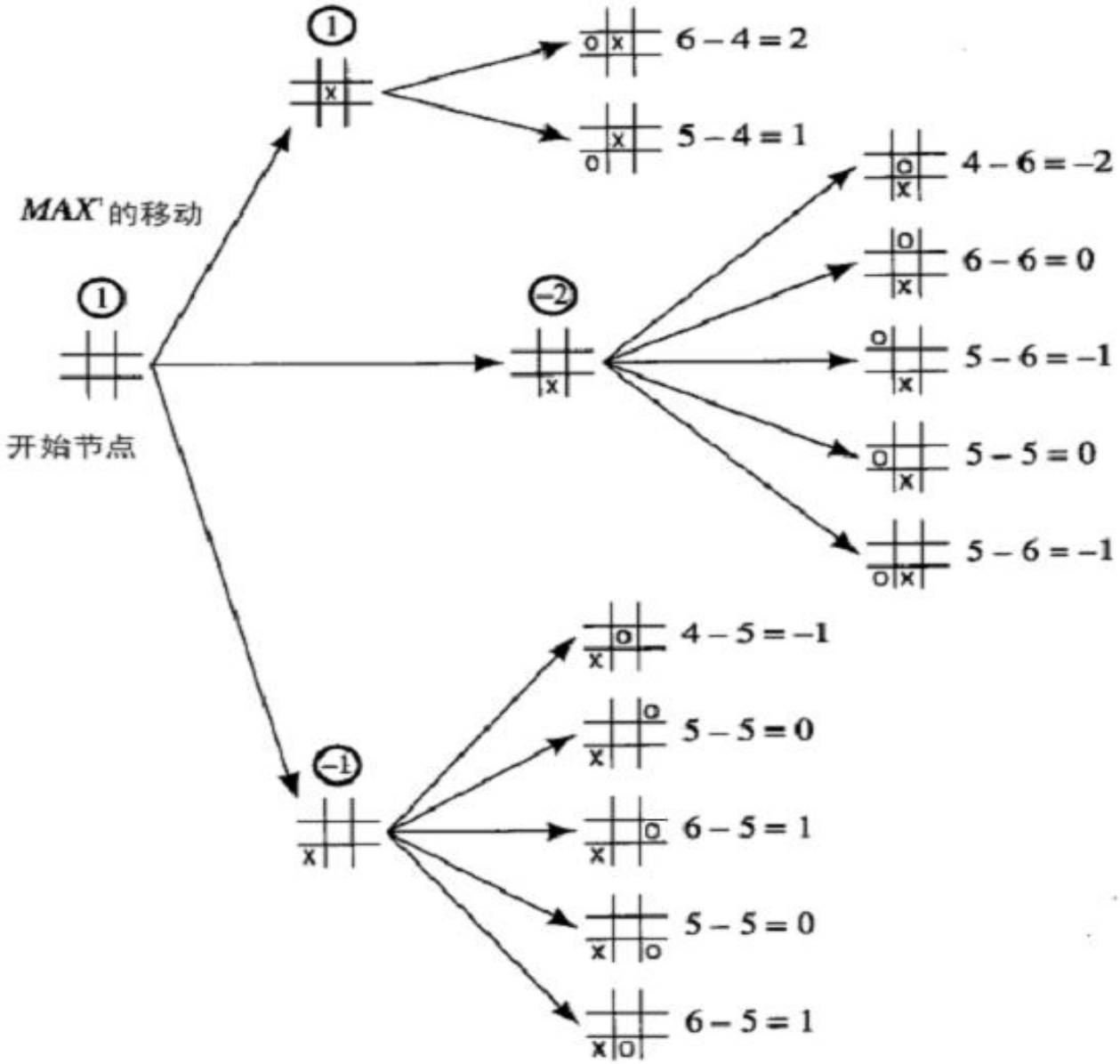


$e(P) = 6 - 4 = 2$

- **Reserving only one for the symmetrical states**

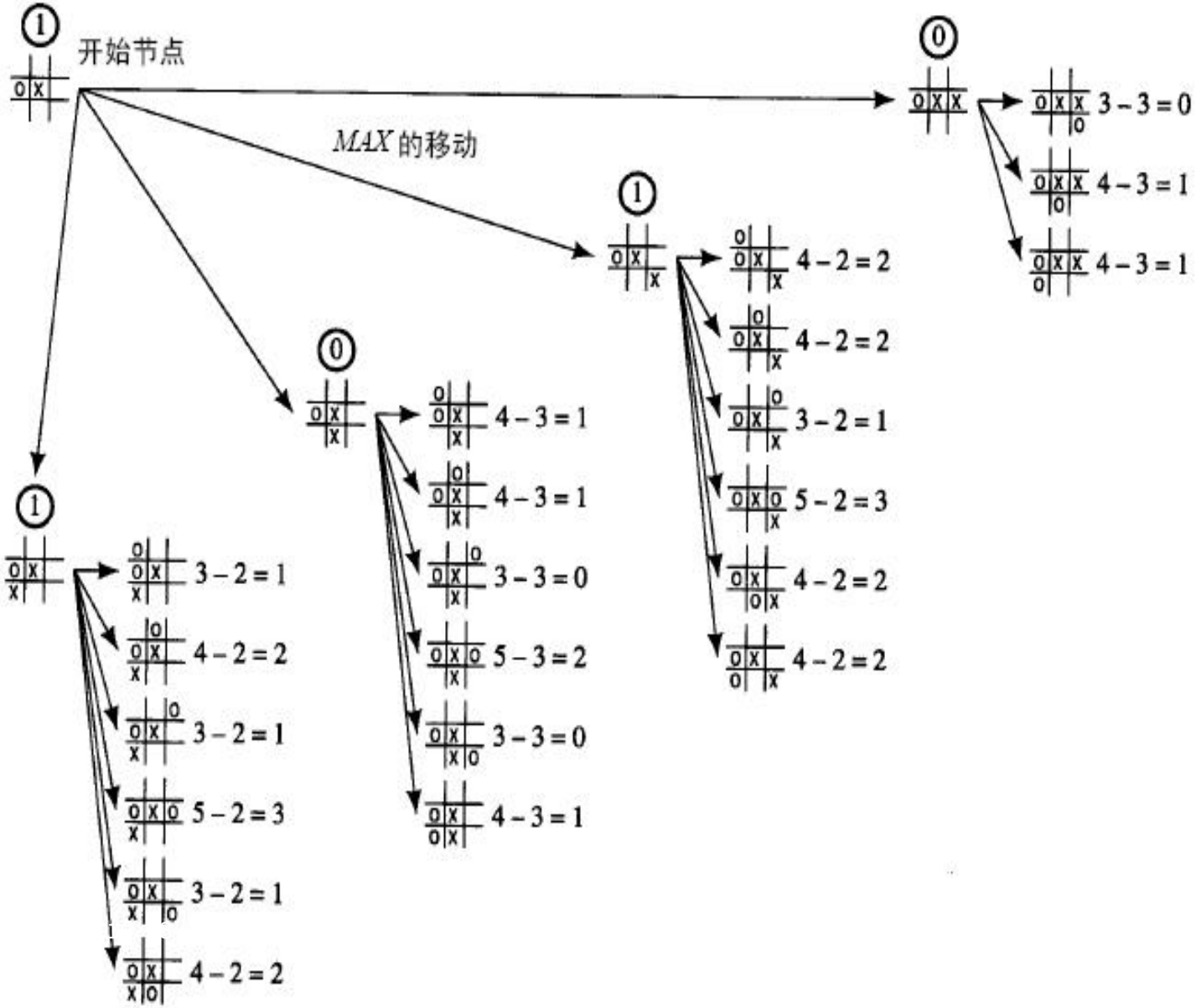





Solution of tic-tac-toe (1)



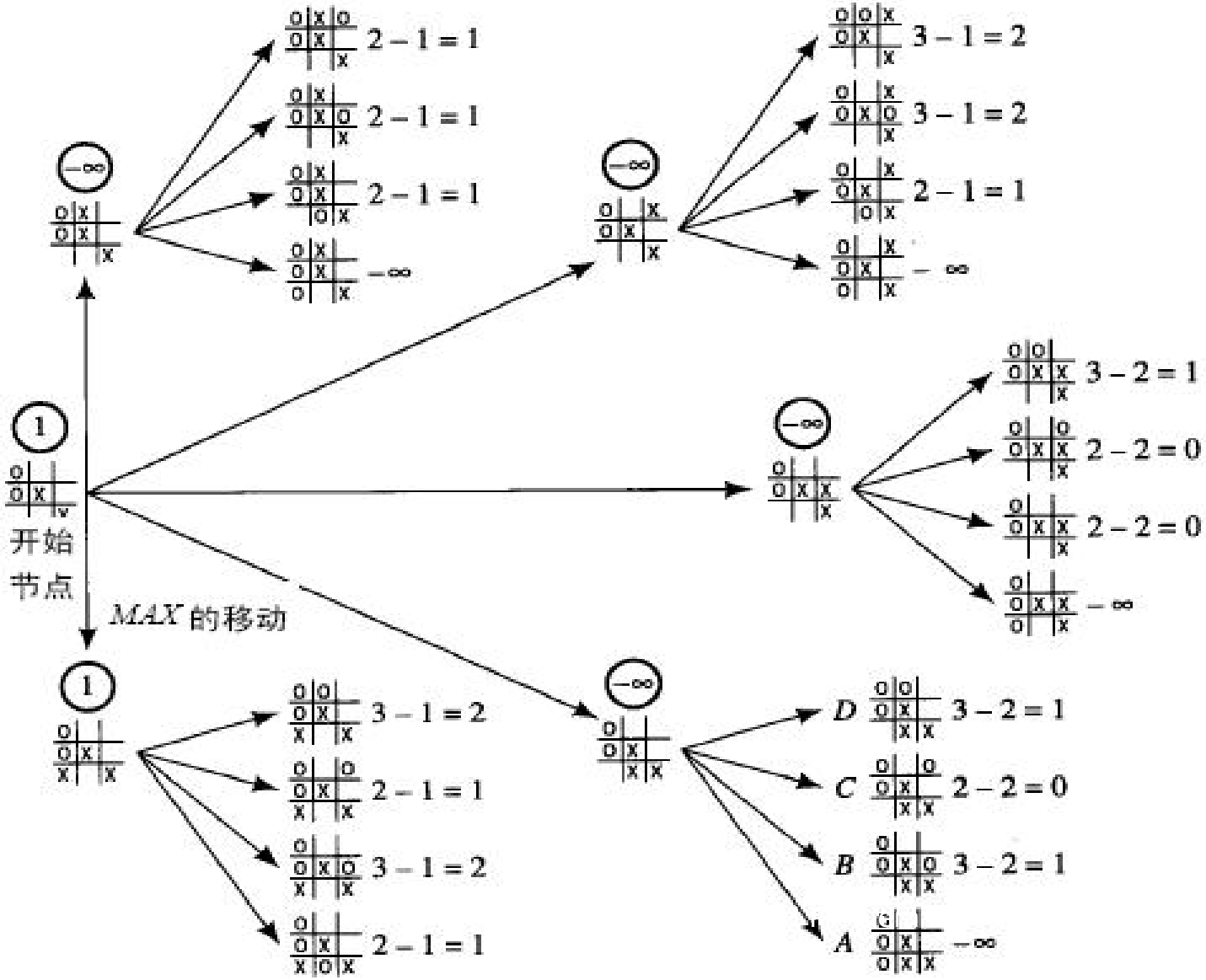
First stage

Solution of tic-tac-toe (2)



Second stage

Solution of tic-tac-toe (3)

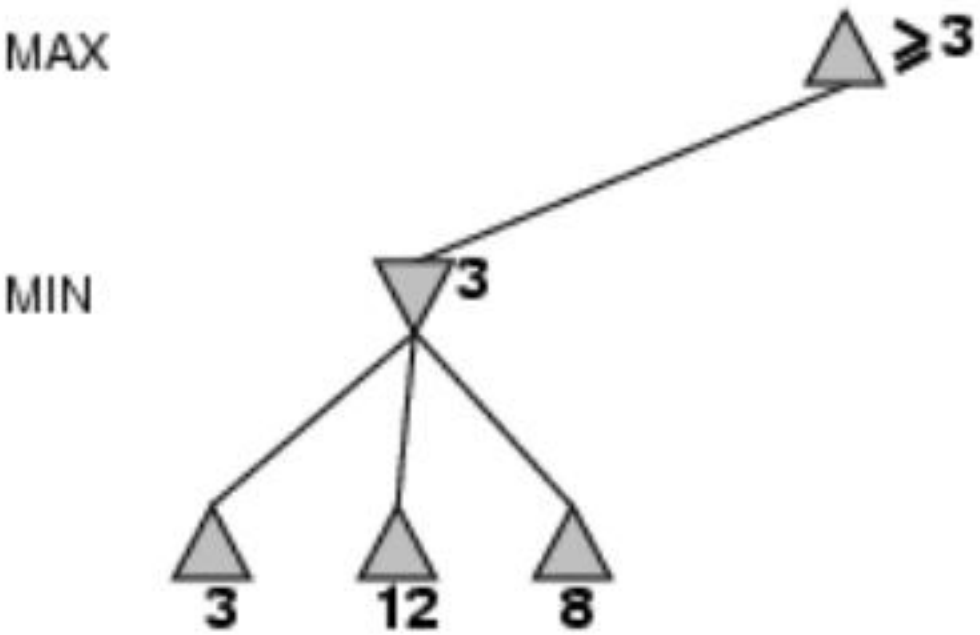


Final stage

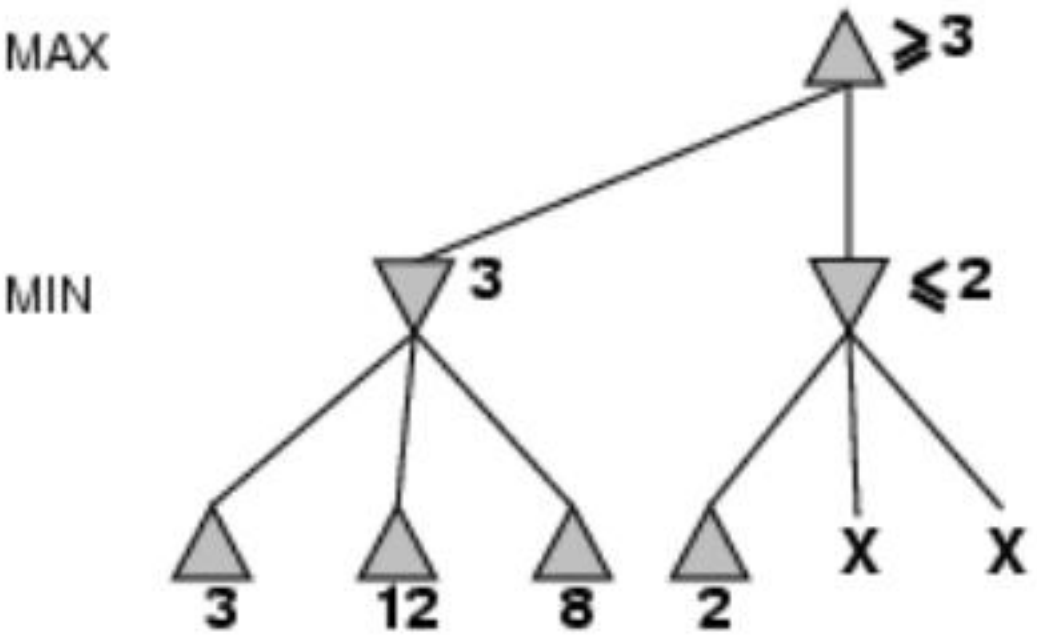
4.3 α-β pruning

- Improve search efficiency through pruning some unnecessary branches in the game tree
- Use depth-limited search (DLS) strategy

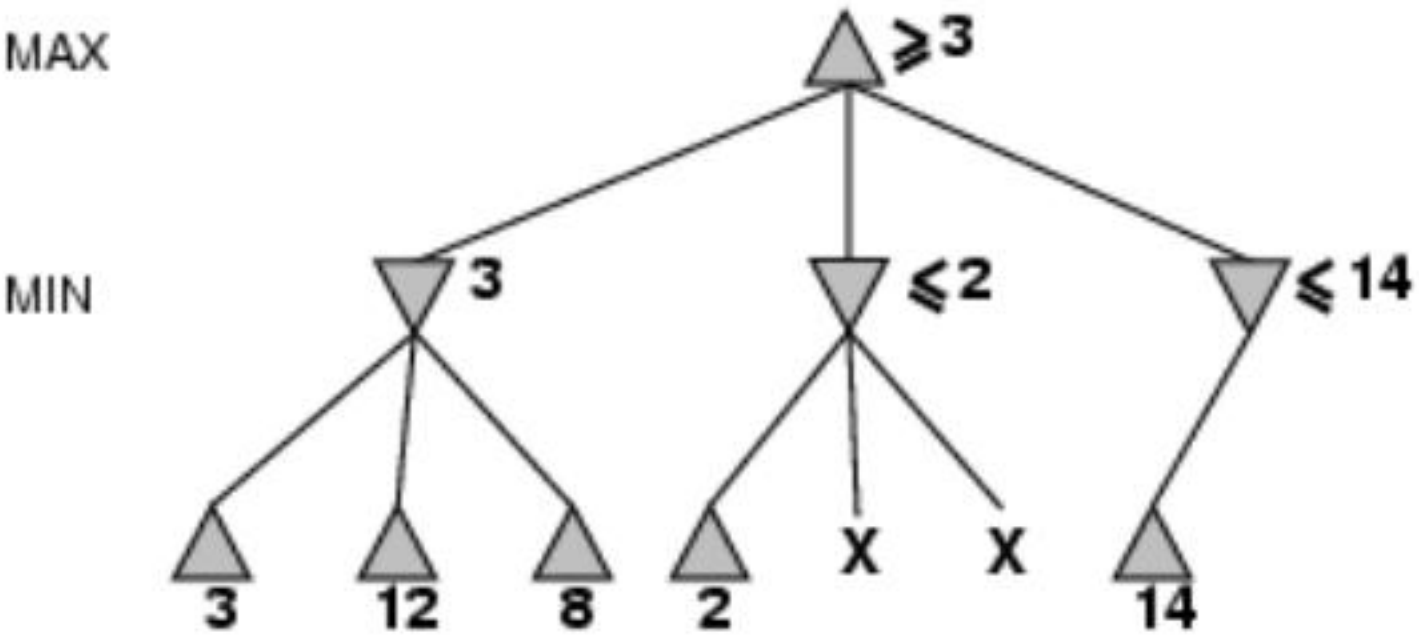
α-β pruning example1



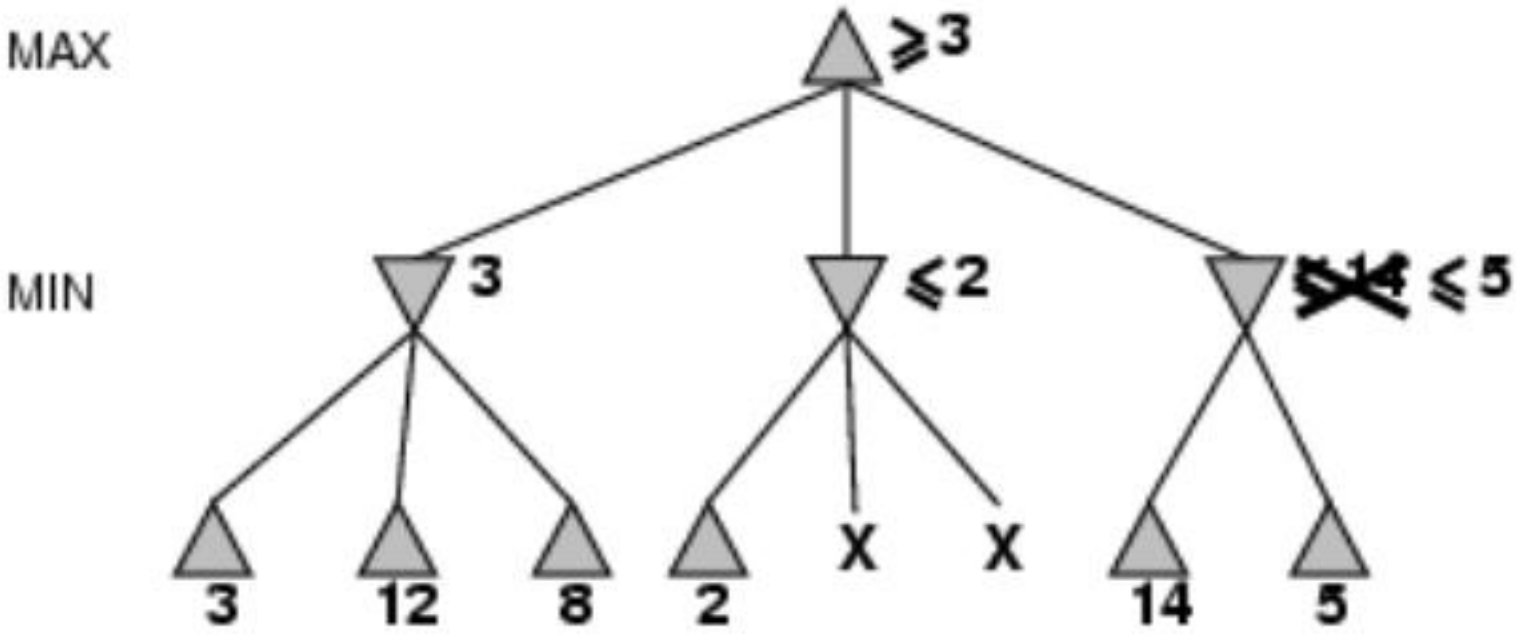
α-β pruning example1



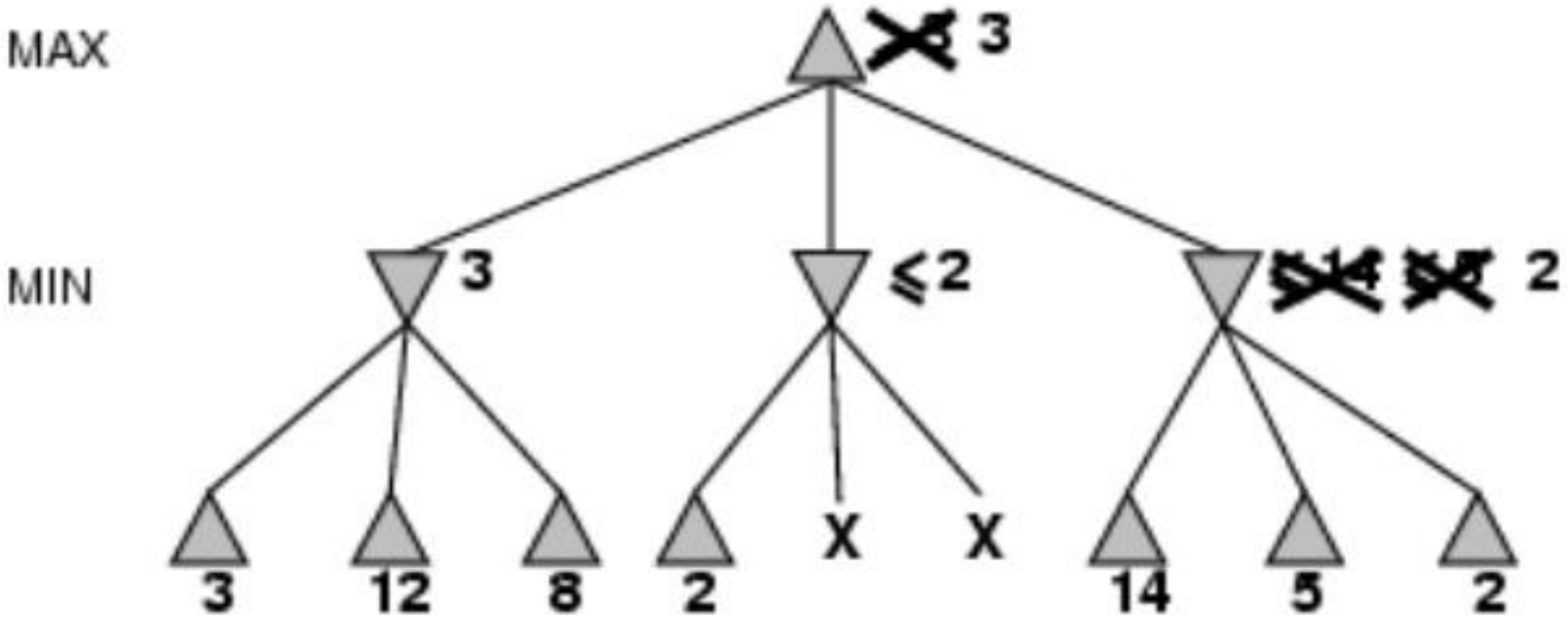
α-β pruning example1



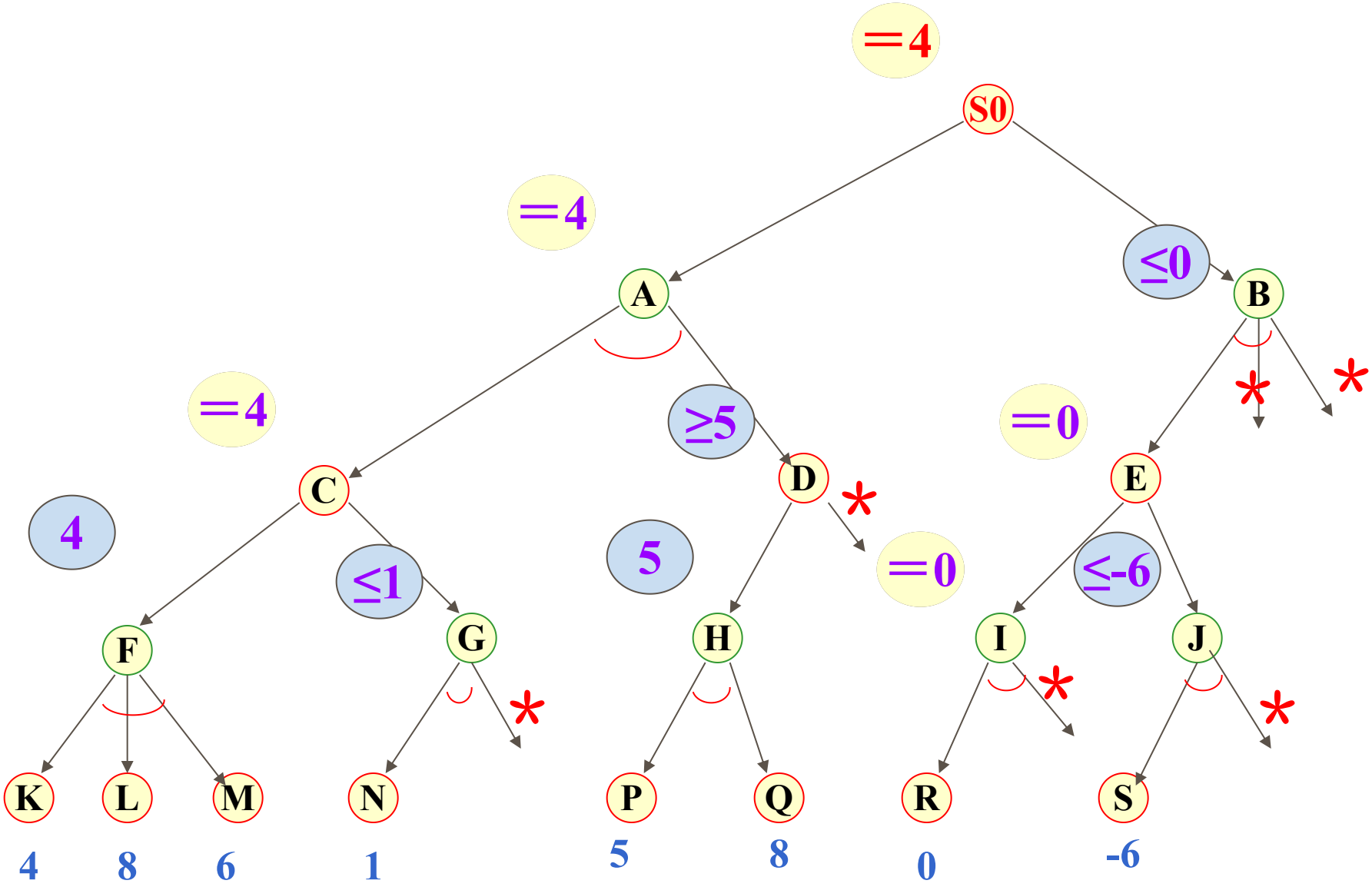
α-β pruning example1



α-β pruning example1

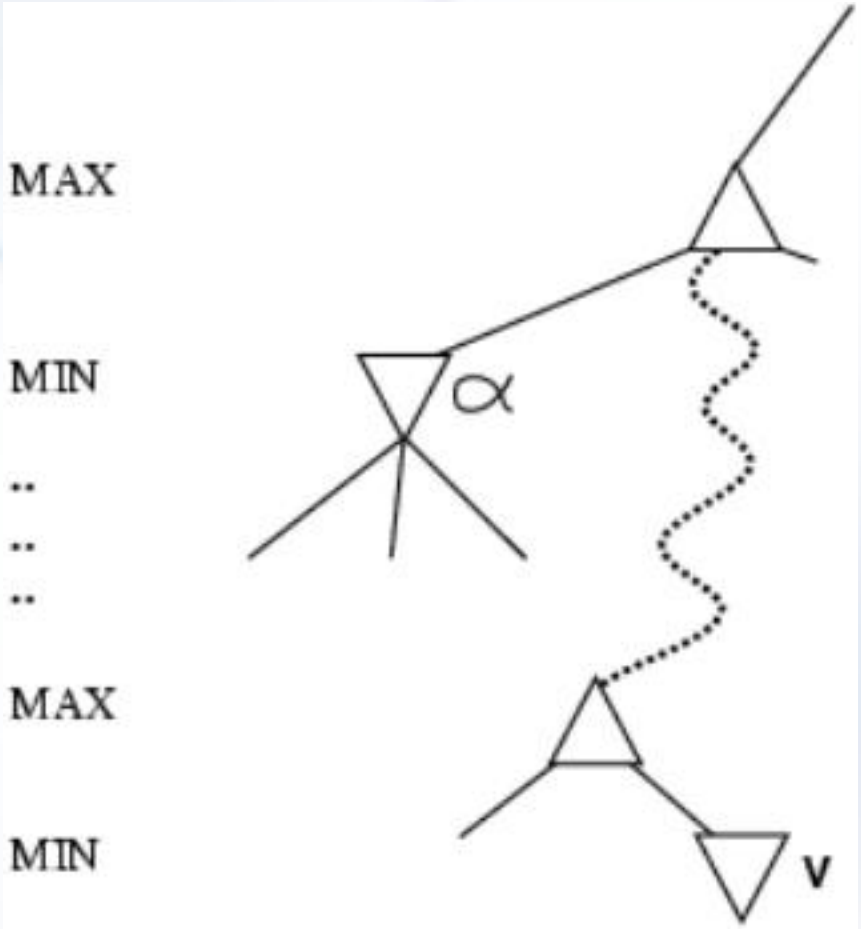


α-β pruning example2



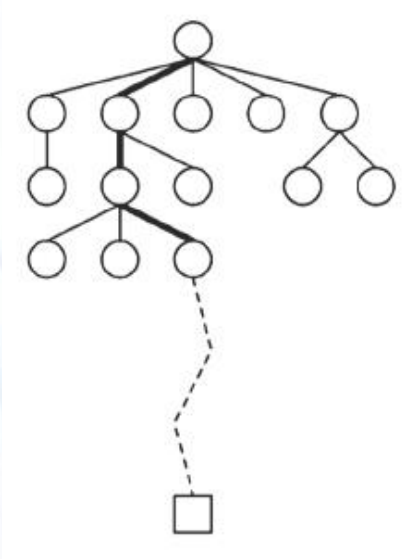
Why is it called α-β?

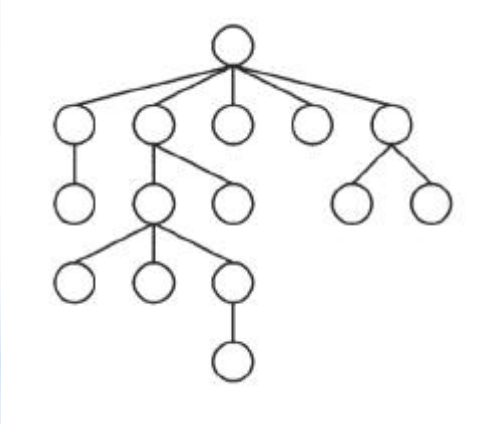
- α is the value of the best (i.e., highest-value) choice found so far at any choice point along the path for MAX
- If *v* is worse than α, MAX will avoid it
  - → prune that branch
- Define β similarly for MIN



4.4 Monte Carlo Tree Search (MCTS)

- No evaluation function!
- Simulate from the leaf node using **random moves**, score the game at the end, use that as evaluation



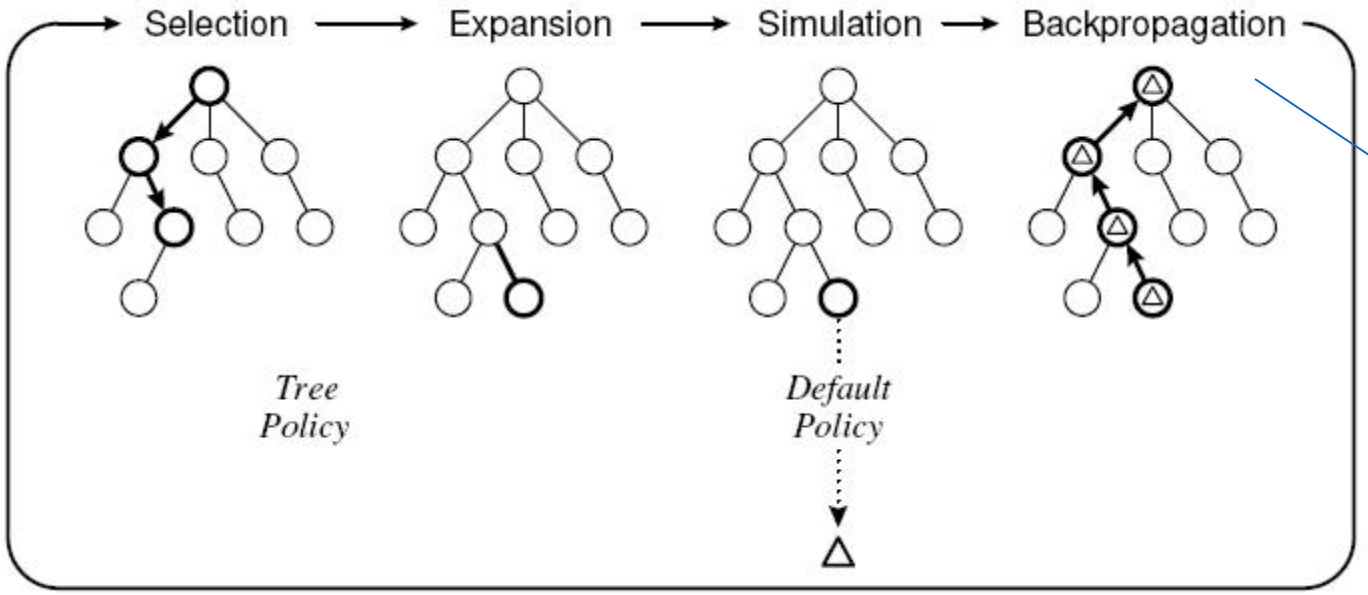


C. Browne et al., A Survey of Monte Carlo Tree Search Methods. IEEE Trans. Computational Intelligence and AI In Games, 2012, 4(1): 1-46

General MCTS Algorithm

- Four steps are applied per search iteration:
  - Selection: a child selection policy is used from the root to reach the most urgent expandable node
  - Expansion: child nodes are added to expand the tree
  - Simulation: run from the new nodes according to the default policy to produce and outcome
  - Backpropagation: the simulation result is backed up to update the statistics of nodes

General MCTS Algorithm



Visit count is incremented and its average reward or Q value updated

Algorithm 1 General MCTS approach.

```
function MCTSSEARCH(s_0)
    create root node v_0 with state s_0
    while within computational budget do
        v_l ← TREEPOLICY(v_0)
        Δ ← DEFAULTPOLICY(s(v_l))
        BACKUP(v_l, Δ)
    return a(BESTCHILD(v_0))
```

Four criteria for selecting the wining action:

- Max child: the highest reward.
- Robust child: the most visited
- Max-Robust child: both the highest visit count and the highest reward
- Secure child: maximises a lower confidence bound.

Upper Confidence Bounds for Trees (UCT)

- Solve the tree policy based on upper confidence bounds (UCB)
- Actually, a child node is selected to maximise:

$$UCT = \overline{X}_j + 2C_p\sqrt{\frac{2\ln n}{n_j}}$$

Xj: the average reward
n: the number of times the current (parent) node has been visited
nj: the number of times child j has been visited
Cp: a constant

UCT algorithm

❖ MCTS with UCB tree selection policy

Algorithm 2 The UCT algorithm.

```
function UCTSEARCH(s_0)
    create root node v_0 with state s_0
    while within computational budget do
        v_l ← TREEPOLICY(v_0)
        Δ ← DEFAULTPOLICY(s(v_l))
        BACKUP(v_l, Δ)
    return a(BESTCHILD(v_0, 0))
```

Each node has four pieces of data: state, action, simulation reword, and visit count

It is proven that given enough time and memory, UCT allows MCTS to converge to the minimax tree

UCT algorithm – details

function TREEPOLICY(v)
  while v is nonterminal **do**
    if v not fully expanded **then**
      return EXPAND(v)
    else
      $v \leftarrow$ BESTCHILD(v, Cp)
  return v

function EXPAND(v)
  choose $a \in$ untried actions from $A(s(v))$
  add a new child v' to v
    with $s(v') = f(s(v), a)$
    and $a(v') = a$
  return v'

function BESTCHILD(v, c)
  return $\arg\max_{v' \in \text{children of } v} \frac{Q(v')}{N(v')} + c\sqrt{\frac{2\ln N(v)}{N(v')}}$

function DEFAULTPOLICY(s)
  while s is non-terminal **do**
    choose $a \in A(s)$ uniformly at random
    $s \leftarrow f(s, a)$
  return reward for state s

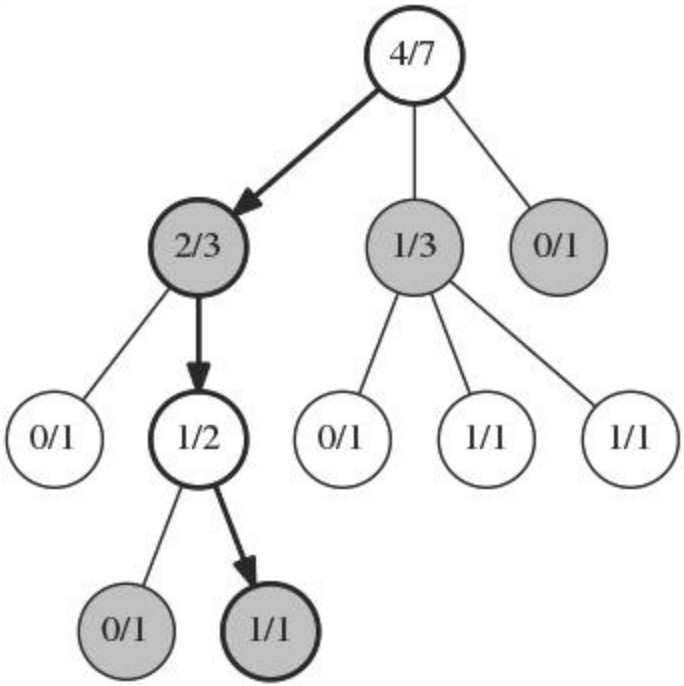
function BACKUP(v, Δ)
  while v is not null **do**
    $N(v) \leftarrow N(v) + 1$
    $Q(v) \leftarrow Q(v) + \Delta(v, p)$
    $v \leftarrow$ parent of v

general

function BACKUPNEGAMAX(v, Δ)
  while v is not null **do**
    $N(v) \leftarrow N(v) + 1$
    $Q(v) \leftarrow Q(v) + \Delta$
    $\Delta \leftarrow -\Delta$
    $v \leftarrow$ parent of v

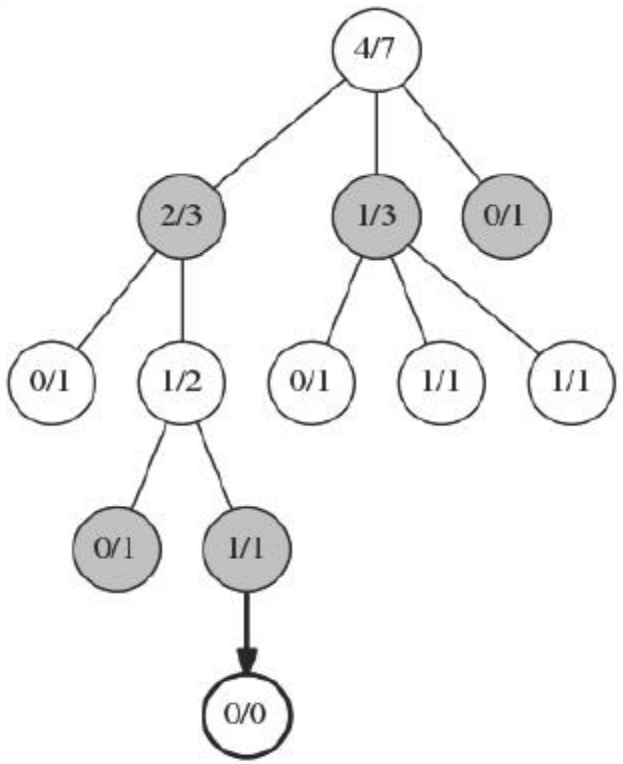
for two-players, zero-sum games

UCT algorithm – An Example



Selection

Here the positions and moves selected by the UCB1 algorithm at each step are marked in bold. Note that a number of playouts have already been run to accumulate the statistics shown. Each circle contains the number of wins / number of times played.

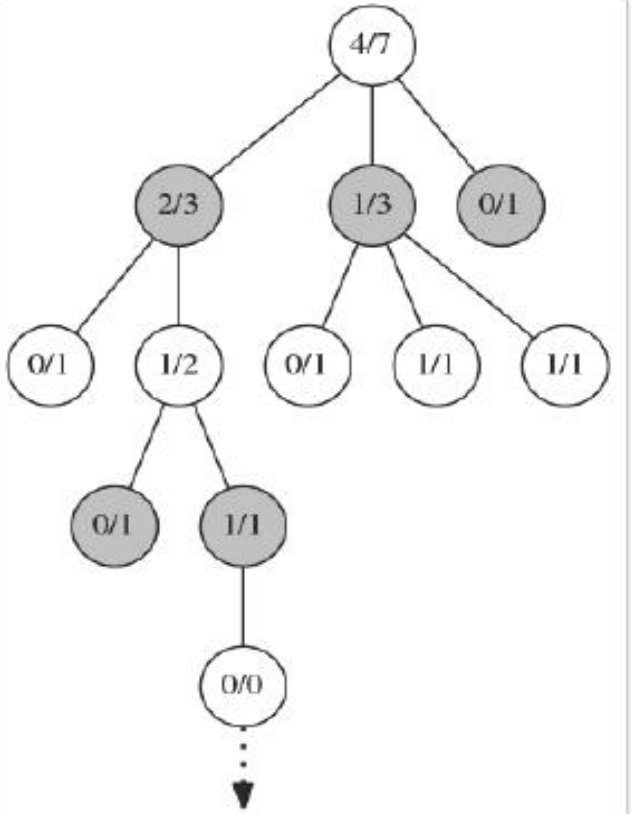


Expansion

The position marked 1/1 at the bottom of the tree has no further statistics records under it, so we choose a random move and add a new record for it (bold), initialized to 0/0.

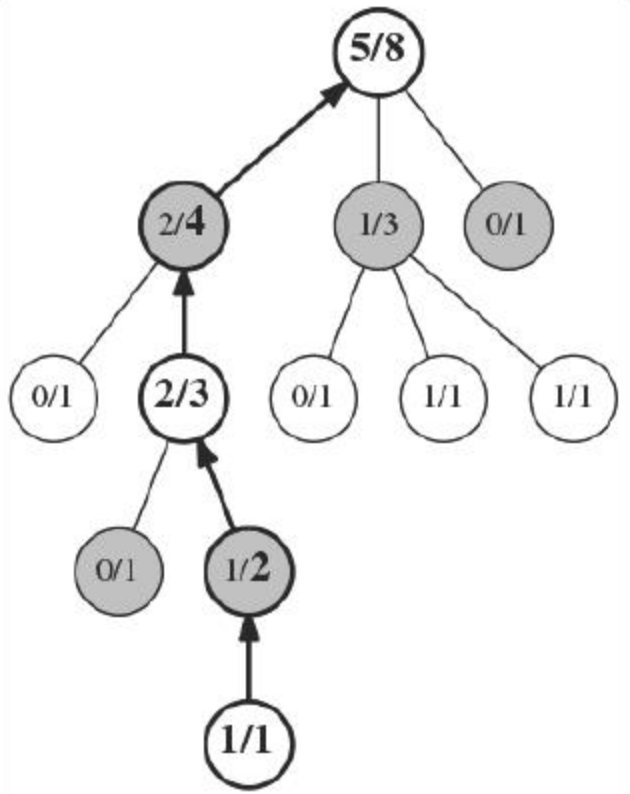
J. Bradberry. Introduction to Monte Carlo Tree Search, 2015

UCT algorithm – An Example



Simulation

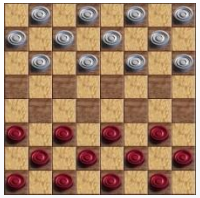
Once the new record is added, the Monte Carlo simulation begins, here depicted with a dashed arrow. Moves in the simulation may be completely random, or may use calculations to weight the randomness in favor of moves that may be better.



Back-Propagation

After the simulation reaches an end, all of the records in the path taken are updated. Each has its play count incremented by one, and each that matches the winner has its win count incremented by one, here shown by the bolded numbers.

Deterministic games in practice



- Checkers: *Chinook* ended 40-year-reign of human world champion Marion Tinsley in 1994.

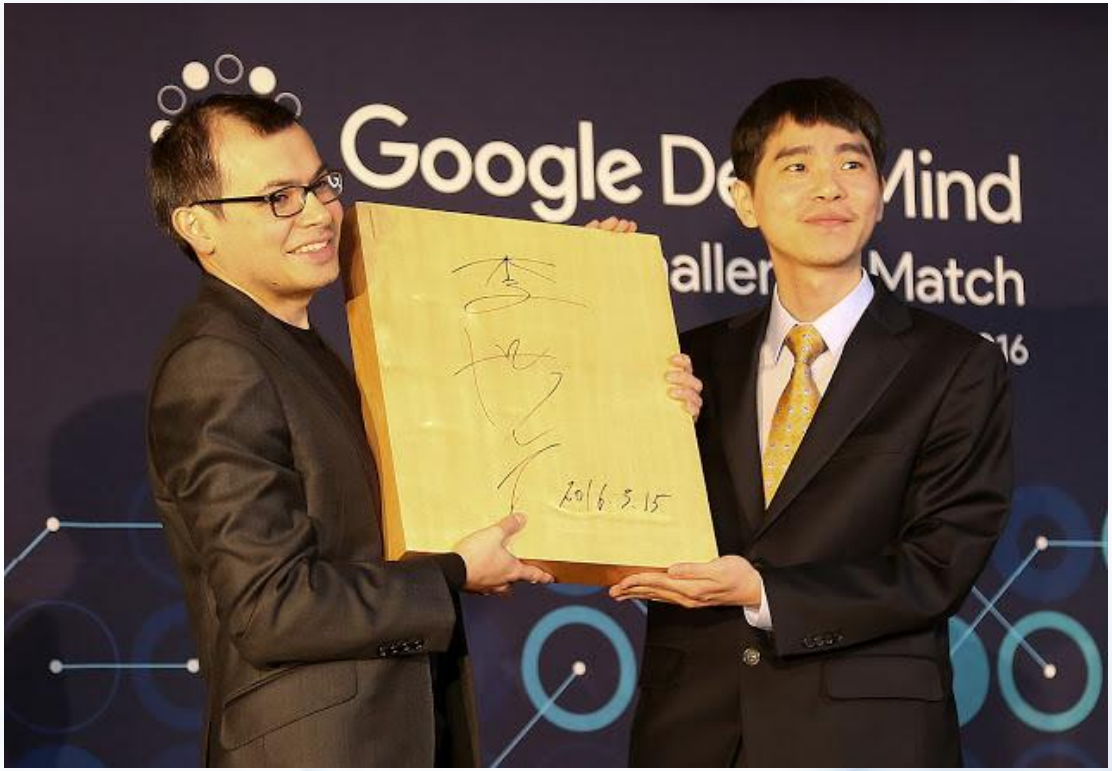


- Chess: *Deep Blue* defeated human world champion *Garry Kasparov* in a six-game match in 1997. Deep Blue searches 200 million positions per second, uses very sophisticated evaluation, and undisclosed methods for extending some lines of search up to 40 ply.



- Go: In March 2016 *AlphaGo* won 4-1 against the legendary *Lee Sedol* , the top Go player in the world over the past decade.

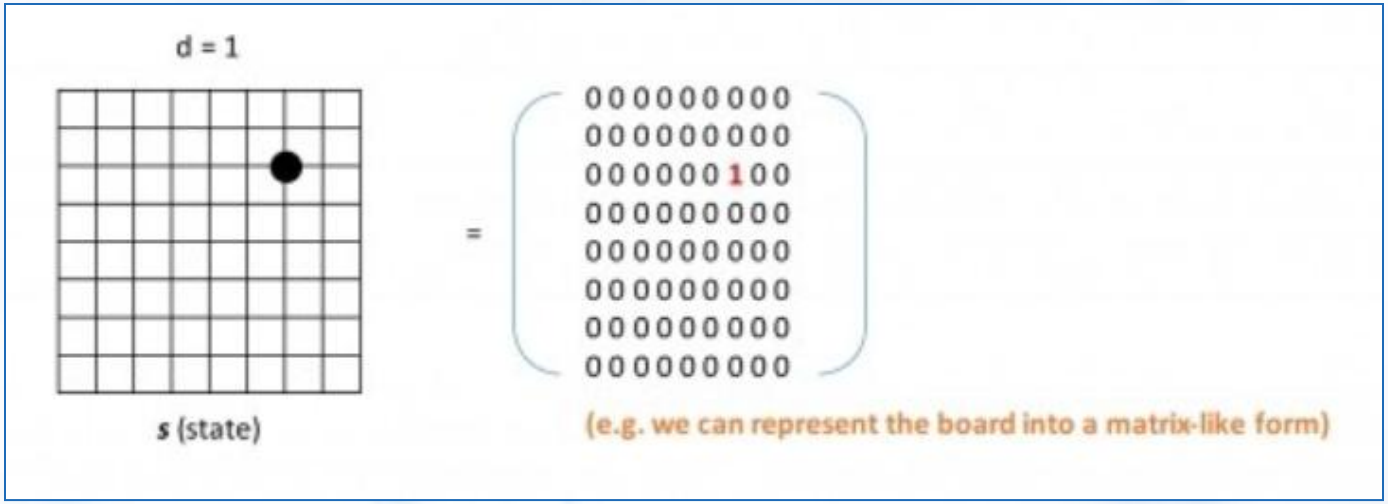
Secret behind AlphaGO

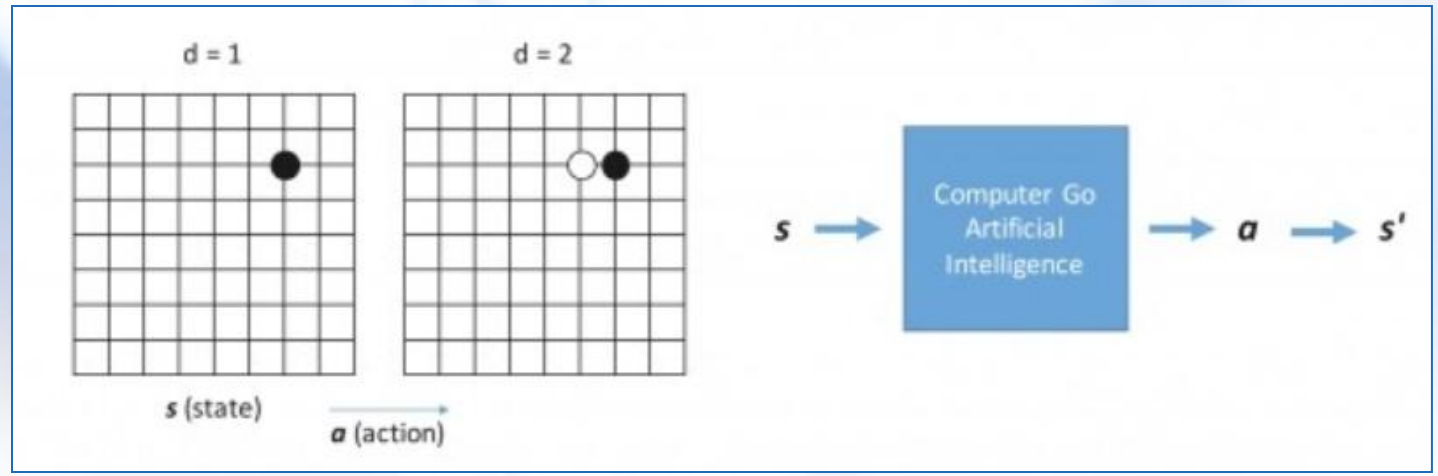


Google Deepmin. Mastering the game of Go with deep neural networks and tree search, Nature, Jan. 2016.

AlphoGO Game Definition

❖ Game State and Action





Key Points

- **Given current game state,**
  - how to decide next action “a”?
  - How to evaluate each legal action?

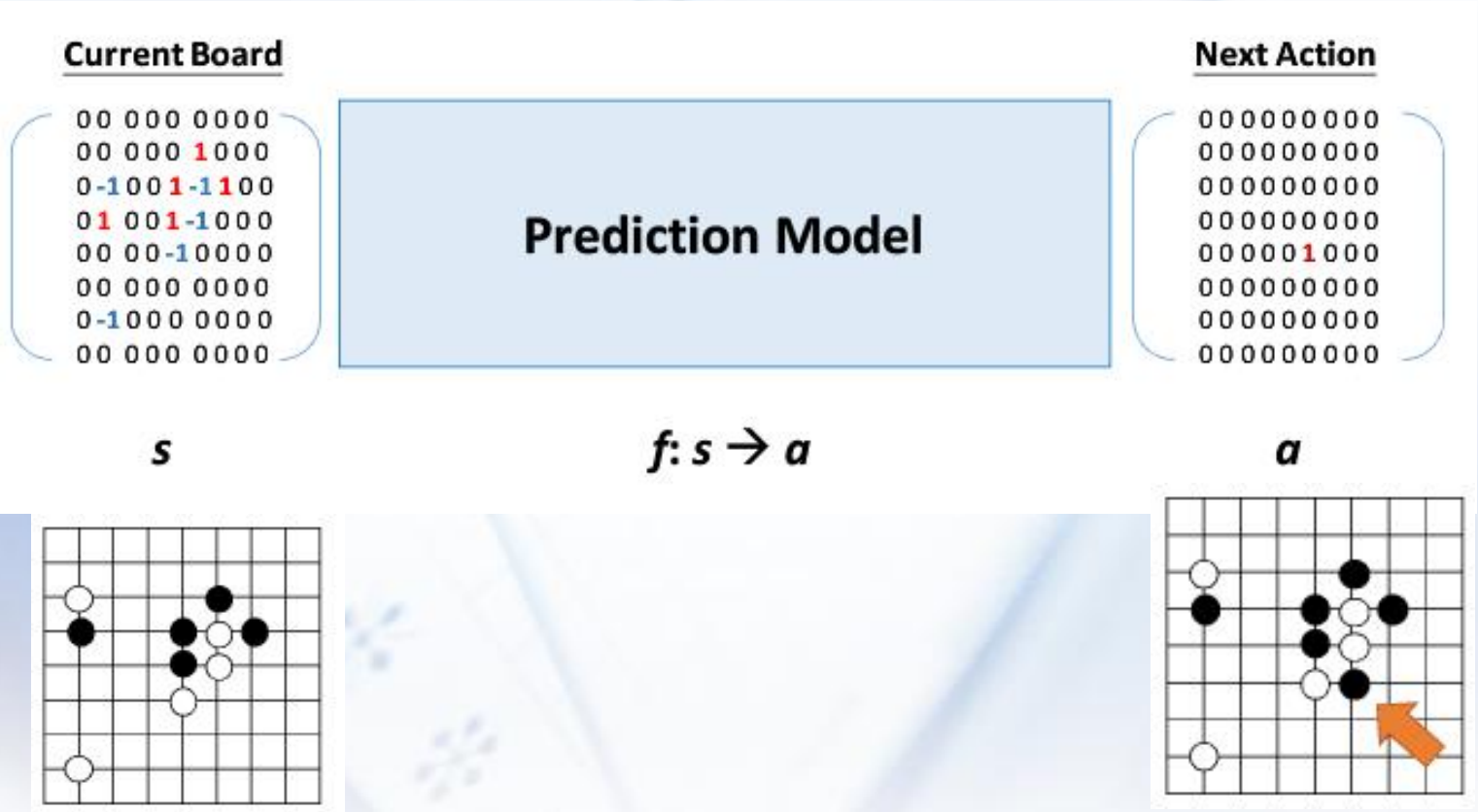
Learning: **P (next action | current state)**

= P (a | s)

- **Method**
  - Imitate Expert Moves (Supervised Learning)
  - Improve through self-plays (Reinforcement Learning)
  - Board Evaluation (Reinforcement Learning)

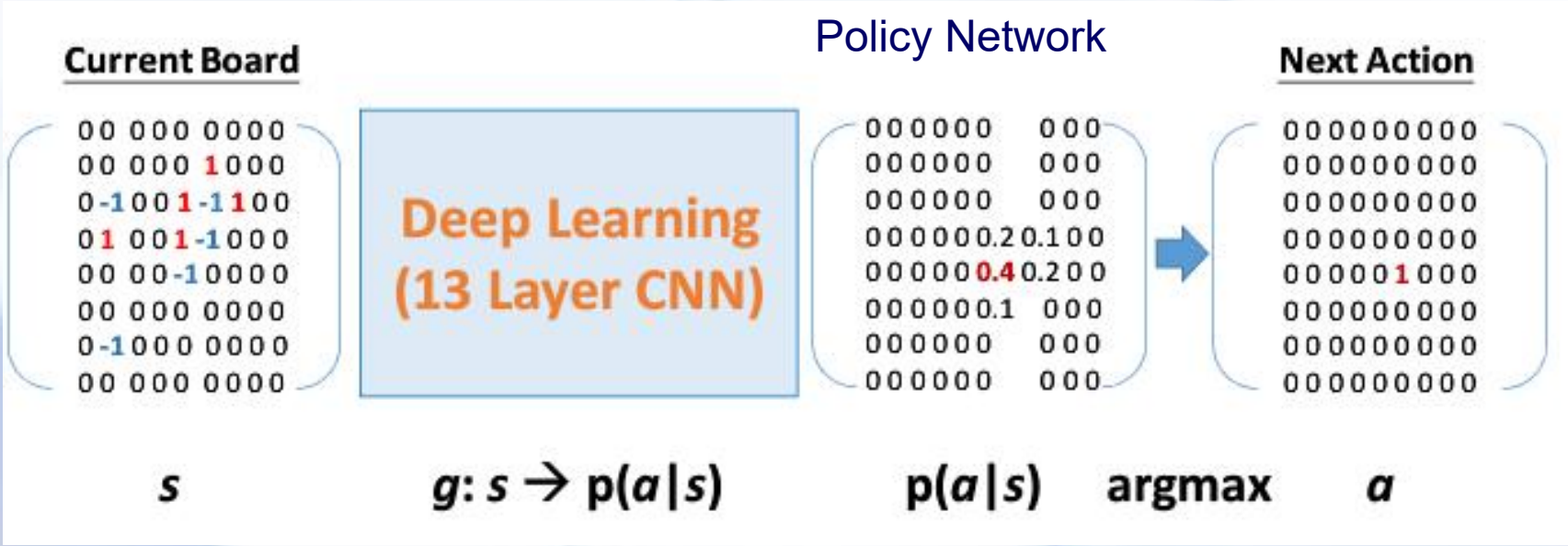
(1) Imitate Expert Moves

- Using supervised learning to obtain a prediction model



Imitate Expert Moves

- Convolutional Neural network (CNN) is used



$$\sigma = \sigma + \eta \frac{p_\sigma(a|s)}{\partial \sigma}$$

Objectiveness: MLE

Optimization: stochastic gradient ascent

(2) Improve through self-plays



Expert Moves Imitator Model (w/ CNN)	VS	Expert Moves Imitator Model (w/ CNN)	Updated Model ver 1.1	VS	Updated Model ver 1.3
Updated Model ver 1.3	VS	Updated Model ver 1.7	Updated Model ver 1.5	VS	Updated Model ver 2.0
Updated Model ver 3204.1	VS	Updated Model ver 46235.2	Expert Moves Imitator Model	VS	Updated Model ver 1,000,000

Improve through self-plays



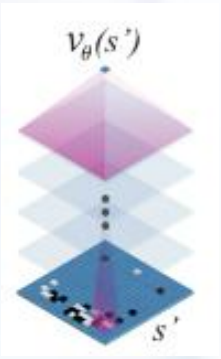
$$\rho = \rho + \eta \frac{\partial \log p_\rho(a_t|s_t)}{\partial \sigma} z_t$$

Objectiveness: MLE

Optimization: stochastic gradient ascent

(3) Board Evaluation

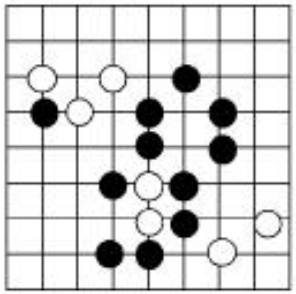




Win / Loss

Win (0~1)

Board Position



Updated Model ver 1,000,000

Value Prediction Model (Regression)

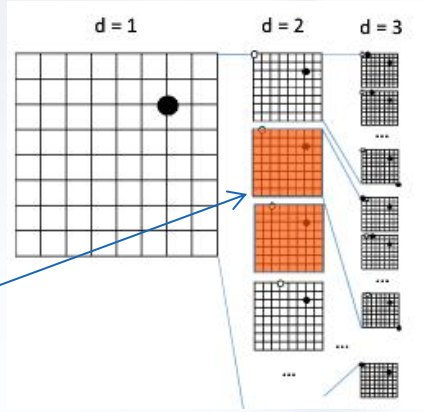
$$\theta = \theta - \eta \frac{\partial v_\theta(s)}{\partial \theta}\left(z - v_\theta(s)\right)$$

Objectiveness: MSE

Optimization: stochastic gradient descent

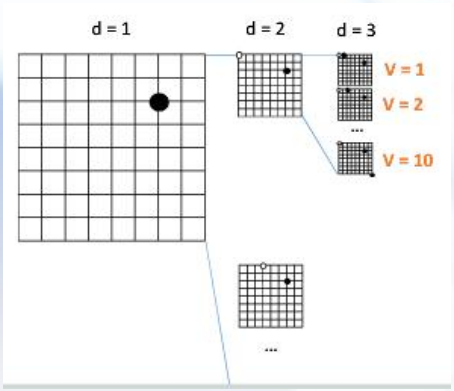
Purpose: Reducing Search Space

- Policy network: reducing action candidates (breadth reduction)



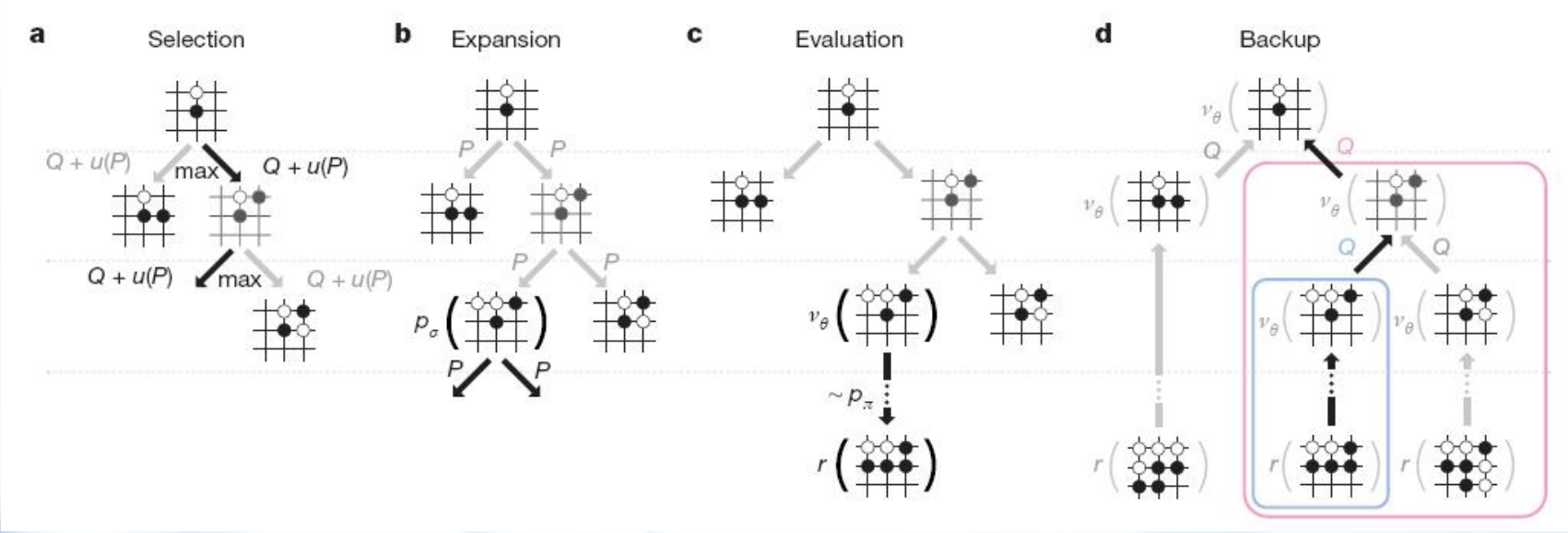
These moves are not common/probable

- Value network (Board Evaluation): depth reduction



Eventually we get these values for evaluating the states

All-in-one: Monte Carlo Tree Search (MCTS)



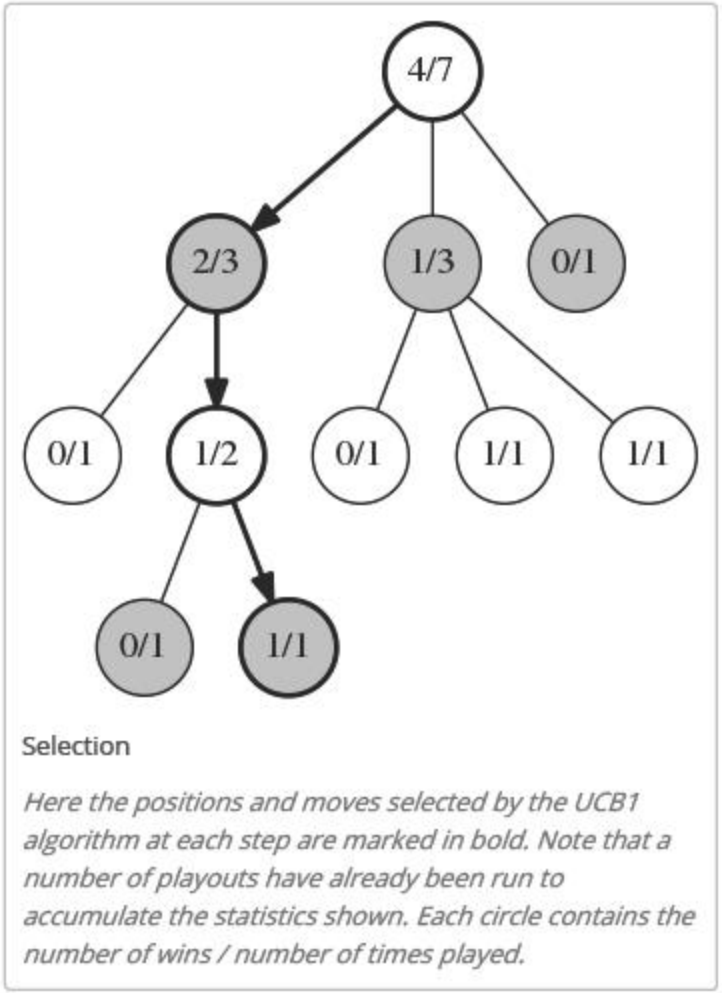
a: Each simulation traverses the tree by selecting the edge with maximum action value Q, plus a bonus u(P) that depends on a stored prior probability P for that edge.

b: The leaf node may be expanded; the new node is processed once by the policy network pσ and the output probabilities are stored as prior probabilities P for each action.

c: At the end of a simulation, the leaf node is evaluated in two ways: using the value network vθ; and by running a rollout to the end of the game with the fast rollout policy pπ, then computing the winner with function r.

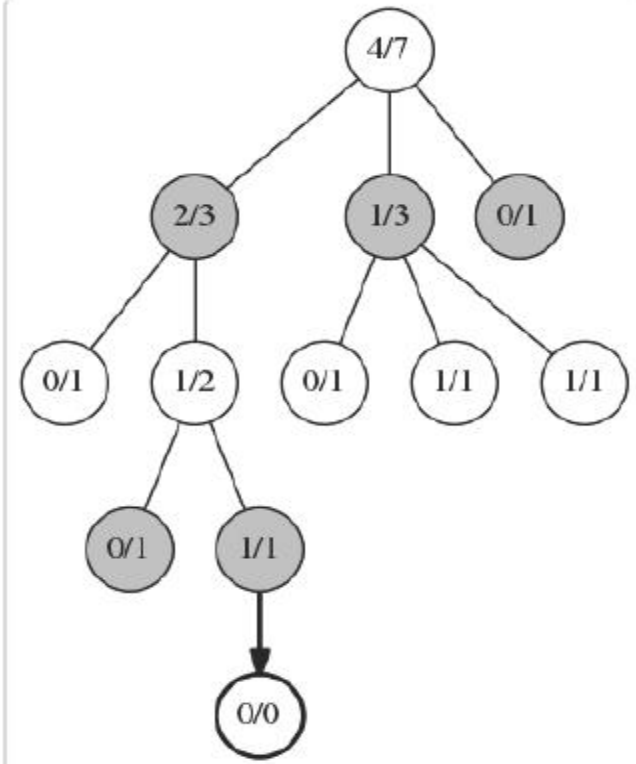
d: Action values Q are updated to track the mean value of all evaluations r(·) and vθ(·) in the subtree below that action.

UCT (MCTS) – An Example



Selection

Here the positions and moves selected by the UCB1 algorithm at each step are marked in bold. Note that a number of playouts have already been run to accumulate the statistics shown. Each circle contains the number of wins / number of times played.

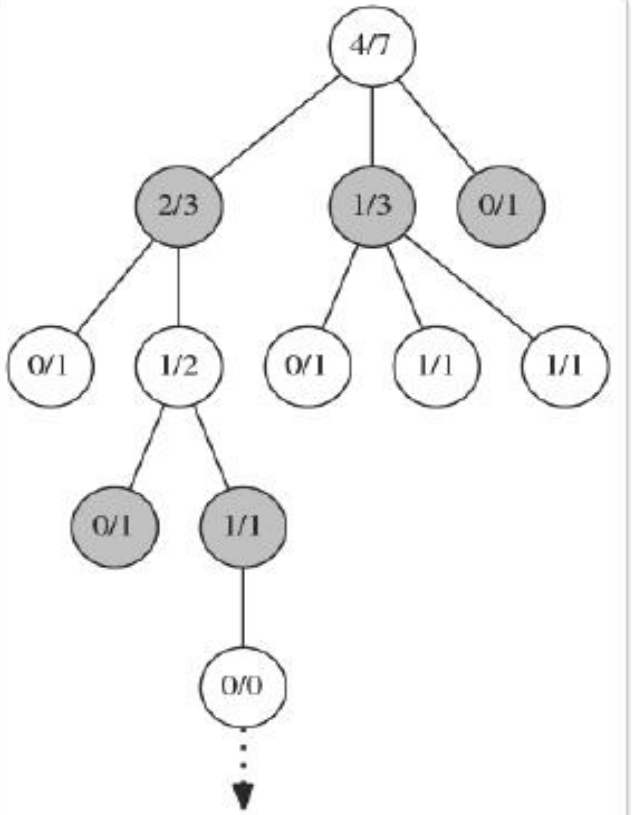


Expansion

The position marked 1/1 at the bottom of the tree has no further statistics records under it, so we choose a random move and add a new record for it (bold), initialized to 0/0.

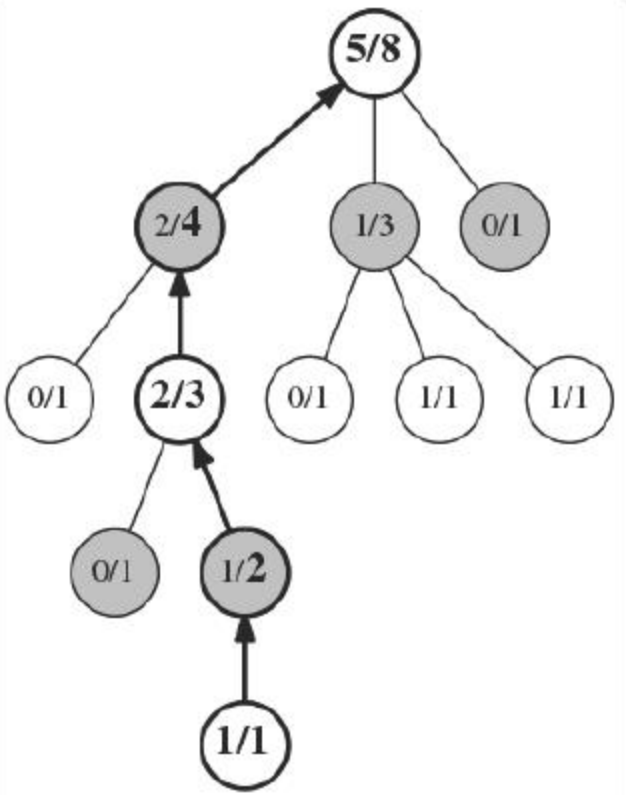
J. Bradberry. Introduction to Monte Carlo Tree Search, 2015

UCT (MCTS) – An Example



Simulation

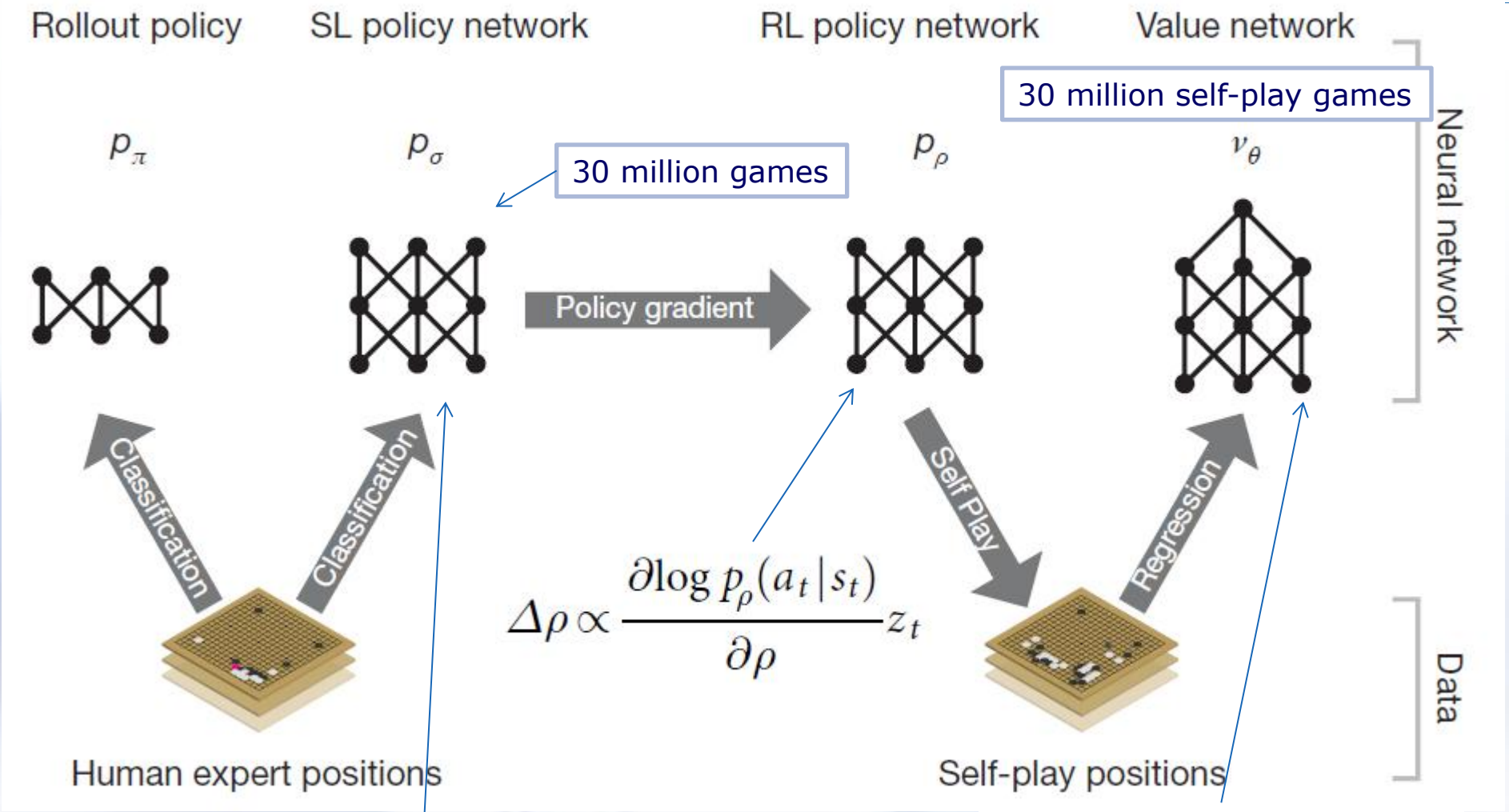
Once the new record is added, the Monte Carlo simulation begins, here depicted with a dashed arrow. Moves in the simulation may be completely random, or may use calculations to weight the randomness in favor of moves that may be better.



Back-Propagation

After the simulation reaches an end, all of the records in the path taken are updated. Each has its play count incremented by one, and each that matches the winner has its win count incremented by one, here shown by the bolded numbers.

Training pipeline



$$\Delta\sigma \propto \frac{\partial \log p_\sigma(a|s)}{\partial \sigma}$$

$$\Delta\theta \propto \frac{\partial v_\theta(s)}{\partial \theta}(z - v_\theta(s))$$

BIT.MLMR

End of Lecture 4

Can you represent and solve the problems by using graph search techniques now?

LIUXIABI@BIT.EDU.CN





